

ÜNİTE 3

ARAMA İLE PROBLEM ÇÖZME

Solving Problems by Searching

Türkçe çeviri + açıklamalı ders notu + şekiller

AIMA 3. Baskı, Bölüm 3 temel alınarak hazırlanmıştır.

Ders notu formatı: kavram tanımları, adım adım örnekler, algoritmalar, karmaşıklıklar ve görsel açıklamalar

Bu doküman, kullanıcı tarafından sağlanan Bölüm 3 metninin Türkçe ders notu biçiminde birleştirilmiş çevirisidir. Kaynak metindeki terminoloji ve bölüm sırası korunmuş; daha önce yapılan açıklamalar, örnek hesaplamalar ve kullanıcının sağladığı şekiller ilgili yerlere eklenmiştir.

İçindekiler

1. 3. Arama ile Problem Çözme
2. 3.1 Problem Çözen Ajanlar
3. 3.1.1 İyi Tanımlanmış Problemler ve Çözümler
4. 3.1.2 Problemlerin Formüle Edilmesi
5. 3.2 Örnek Problemler
6. 3.2.1 Oyuncak Problemler
7. 3.2.2 Gerçek Dünya Problemleri
8. 3.3 Çözüm Arama
9. 3.3.1 Arama Algoritmalarının Altyapısı
10. 3.3.2 Problem Çözme Performansının Ölçülmesi
11. 3.4 Bilgisiz Arama Stratejileri
12. 3.4.1 Genişlik Öncelikli Arama (BFS)
13. 3.4.2 Tekdüzen Maliyetli Arama (UCS)
14. 3.4.3 Derinlik Öncelikli Arama (DFS)
15. 3.4.4 Derinlik Sınırlı Arama (DLS)
16. 3.4.5 Yinelemeli Derinleşen Arama (IDS)
17. 3.4.6 Çift Yönlü Arama
18. 3.4.7 Bilgisiz Arama Stratejilerinin Karşılaştırılması
19. 3.5 Bilgili (Sezgisel) Arama Stratejileri
20. 3.5.1 Greedy Best-First Search
21. 3.5.2 A* Araması
22. 3.5.3 Bellek Sınırlı Sezgisel Arama
23. 3.5.4 Daha İyi Aramayı Öğrenmek
24. 3.6 Sezgisel Fonksiyonlar
25. 3.6.1 Heuristic Doğruluğunun Performansa Etkisi
26. 3.6.2 Gevşetilmiş Problemlerden Heuristic Üretmek
27. 3.6.3 Pattern Database
28. 3.6.4 Deneyimden Heuristic Öğrenme

3. ARAMA İLE PROBLEM ÇÖZME

Bu bölümde, tek bir eylemin yeterli olmadığı durumlarda bir ajanın hedefine ulaşmasını sağlayacak eylem dizisini nasıl bulabileceğini inceliyoruz. Bölüm 2'deki refleks ajanlar, durumdan eyleme doğrudan eşleme kullanıyordu. Böyle bir eşleme çok büyükse saklamak veya öğrenmek zorlaşır. Hedef tabanlı ajanlar ise gelecekteki eylemleri ve bu eylemlerin sonuçlarını dikkate alır.

Bu bölümde ele alınan özel hedef tabanlı ajan türüne problem çözen ajan (problem-solving agent) denir. Bu ajanlar atomik gösterimler kullanır: dünya durumları, problem çözme algoritmasının görebildiği iç yapısı olmayan bütünler olarak ele alınır. Daha gelişmiş parçalı veya yapılandırılmış gösterimler kullanan hedef tabanlı ajanlara genellikle planlama ajanları denir.

Önce problem ve çözüm kavramlarını kesin biçimde tanımlayacağız. Ardından genel amaçlı arama algoritmalarına geçeceğiz. Bilgisiz arama yöntemleri problem tanımı dışında ek yönlendirme almaz; bilgili arama yöntemleri ise hangi yönlerin daha umut verici olduğu konusunda sezgisel bilgi kullanır.

Bölümün temel fikri

Bir problemin çözümü, başlangıç durumundan hedef duruma götüren sabit bir eylem dizisi olarak düşünülebilir. Bu bölümde ortamın gözlemlenebilir, ayrık, bilinen ve deterministik olduğu basit görev ortamına odaklanıyoruz.

3.1 PROBLEM ÇÖZEN AJANLAR

Akıllı ajanların amacı performans ölçütünü mümkün olduğunca yükseltmektir. Bunu yapmanın bir yolu, mevcut duruma göre bir hedef benimsemek ve davranışı bu hedef doğrultusunda düzenlemektir.

Arad - Bükreş örneği

Romanya'nın Arad şehrinde gezi yapan bir ajan düşünelim. Normalde güneşlenmek, Romence geliştirmek, turistik yerleri görmek, gece hayatını değerlendirmek gibi birçok amacı olabilir. Fakat ertesi gün Bükreş'ten kalkacak, iade edilmeyen bir uçak bileti varsa en mantıklı hedef Bükreş'e ulaşmaktır. Böylece Bükreş'e zamanında ulaştırmayan eylemler elenebilir ve karar problemi önemli ölçüde basitleşir.

Hedef formülasyonu (Goal Formulation)

Hedefler, ajanın aynı anda değerlendirmek zorunda olduğu amaçları sınırlar. Hedef formülasyonu; mevcut durum ve performans ölçütü dikkate alınarak hangi dünya durumlarının başarı sayılacağına belirlenmesidir. Romanya örneğinde hedef kümesi tek bir durumdan oluşur:

$$Goal = \{ In(Bucharest) \}$$

Problem formülasyonu (Problem Formulation)

Ajan hedefi belirledikten sonra hangi durumları ve hangi eylemleri dikkate alacağını seçmelidir. Gerçek dünyayı "direksiyonu bir derece döndür", "sol ayağı bir santimetre ileri taşı" gibi aşırı ayrıntılı eylemlerle modellemek çözümü gereksiz biçimde büyütür. Uygun soyutlama düzeyinde her eylem bir büyük şehirden diğerine sürüş olarak ele alınabilir; her durum da belirli bir şehirde bulunmayı temsil eder.

Arad'dan doğrudan Sibiu, Timisoara ve Zerind'e gidilebilir. Hiçbiri hedef değildir. Ajan Romanya coğrafyasını bilmiyorsa hangi seçimin daha iyi olduğunu doğrudan bilemez. Harita, olası gelecekteki durumlar ve eylemler hakkında bilgi sağlayarak ajanın Bükreş'e giden bir yol aramasına olanak verir.

Bu bölümdeki ortam varsayımları

Gözlemlenebilir: ajan hangi durumda olduğunu bilir. Ayrık: her durumda sonlu sayıda eylem vardır. Bilinen: ajan her eylemin hangi duruma götüreceğini bilir. Deterministik: her eylemin tam olarak bir sonucu vardır.

Arama, çözüm ve yürütme

Hedefe ulaştıran eylem dizisini bulma işlemine arama (search) denir. Bir arama algoritması problemi girdi olarak alır ve çözümü eylem dizisi biçiminde döndürür. Çözüm bulunduktan sonra bu eylemler sırayla uygulanır; buna yürütme (execution) aşaması denir. Böylece problem çözen ajanın temel döngüsü “formüle et - ara - uygula” biçimindedir.

Formulate -> Search -> Execute

Ajan çözüm dizisini yürütürken çevreden gelen yeni algıları dikkate almadan planı aynen uyguluyorsa buna açık döngü (open-loop) davranışı denir. Bu ancak çevrenin yeterince güvenilir ve deterministik olması durumunda makuldür.

3.1.1 İyi Tanımlanmış Problemler ve Çözümler

Bir problem resmî olarak beş bileşenle tanımlanır:

Bileşen	Anlamı	Romanya örneği
Başlangıç durumu	Ajanın başladığı durum	In(Arad)
Eylemler	Bir durumda uygulanabilir eylemler	Go(Sibiu), Go(Timisoara), Go(Zerind)
Geçiş modeli	Bir eylemin hangi yeni durumu oluşturduğu	RESULT(In(Arad), Go(Zerind)) = In(Zerind)
Hedef testi	Bir durumun hedef olup olmadığını sınar	In(Bucharest)?
Yol maliyeti	Bir yolun sayısal maliyeti	Gidilen kilometrelerin toplamı

Başlangıç durumu

Ajanın problemi çözmeye başladığı durumdur. Romanya örneğinde başlangıç In(Arad) olarak gösterilir.

Eylemler ve uygulanabilirlik

ACTIONS(s), s durumunda gerçekleştirilebilecek eylemleri döndürür. Arad için uygulanabilir eylemler Sibiu, Timisoara ve Zerind'e gitmektir.

Geçiş modeli ve ardıl durum

RESULT(s,a), s durumunda a eylemi yapıldığında oluşan durumu döndürür. Tek bir eylemle ulaşılabilen duruma ardıl (successor) denir.

$$RESULT(In(Arad), Go(Zerind)) = In(Zerind)$$

Durum uzayı ve yollar

Başlangıçtan herhangi bir eylem dizisiyle ulaşılabilen tüm durumların kümesi durum uzayıdır. Bu uzay yönlü bir grafik olarak gösterilebilir: düğümler durumları, bağlantılar eylemleri temsil eder. Bir yol, eylemler aracılığıyla bağlanan durumlar dizisidir.

Hedef testi

Hedef testi verilen durumun hedef kümesine ait olup olmadığını belirler. Romanya probleminde hedef kümesi {In(Bucharest)}'tir. Bazı problemlerde hedef tek bir durum değil, “şah mat” gibi soyut bir özellik olabilir.

Yol maliyeti ve optimal çözüm

Yol maliyeti, yolun sayısal değerini verir. Bu bölümde toplam yol maliyetinin tek tek adım maliyetlerinin toplamı olduğu varsayılır. Bir adımın maliyeti $c(s,a,s')$ ile gösterilir ve negatif değildir. Tüm çözümler arasında en düşük yol maliyetine sahip olan çözüm optimal çözümdür.

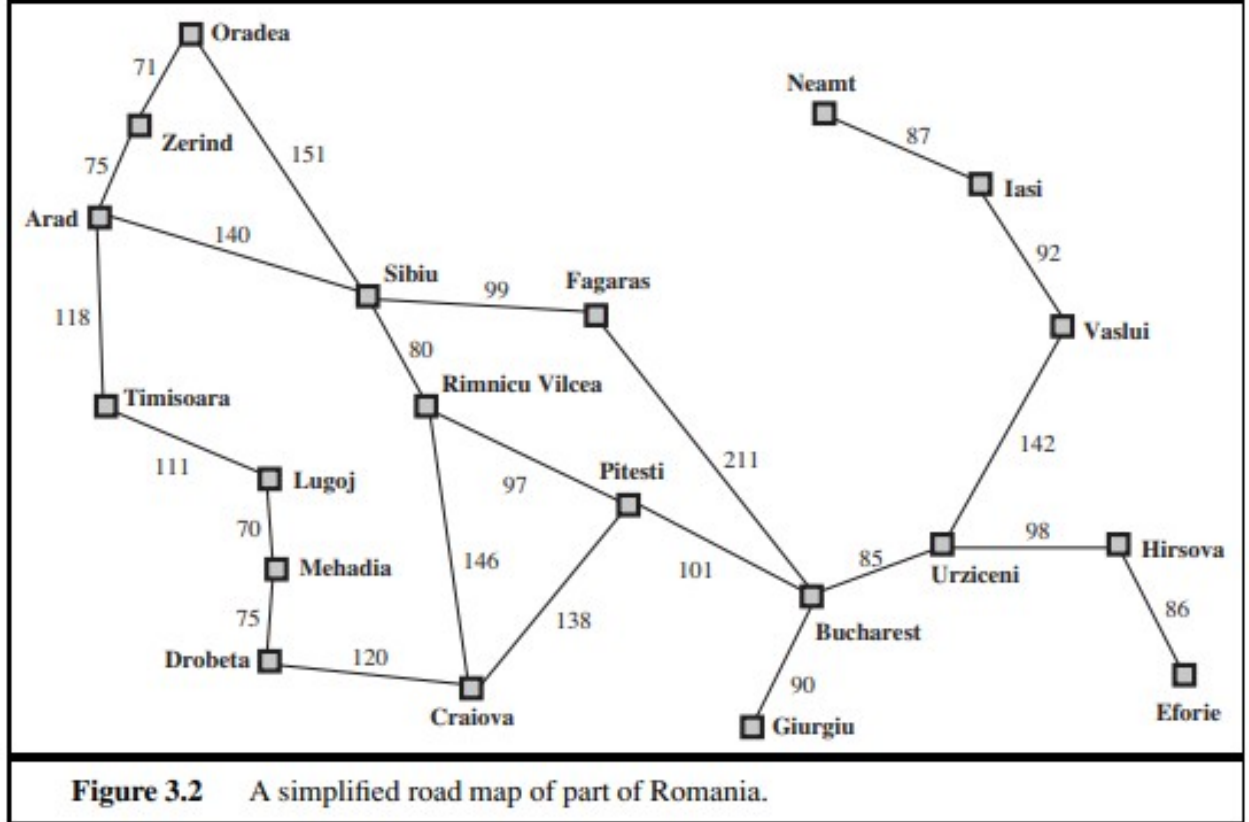
$$PathCost = \sum c(s, a, s')$$

```

function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  state <- UPDATE-STATE(state, percept)
  if seq is empty then
    goal <- FORMULATE-GOAL(state)
    problem <- FORMULATE-PROBLEM(state, goal)
    seq <- SEARCH(problem)
    if seq = failure then return null action
  action <- FIRST(seq)
  seq <- REST(seq)
  return action

```

Şekil 3.1'in özeti: ajan hedef ve problem tanımlar, bir çözüm dizisi arar ve bu dizinin eylemlerini tek tek uygular. Dizi bittiğinde yeni bir hedef belirler.



Şekil 3.2 - Romanya'nın bir bölümünün basitleştirilmiş yol haritası. Şehirler durumları, yollar eylemleri, sayıların her biri ise yol maliyetini (km) gösterir.

Örneğin Arad - Sibiu - Rimnicu Vilcea - Pitesti - Bucharest yolunun maliyeti $140 + 80 + 97 + 101 = 418$ km'dir. Arad - Sibiu - Fagaras - Bucharest yolu ise $140 + 99 + 211 = 450$ km'dir. Bu yüzden kilometre maliyet ölçütüyse 418 km'lik yol daha iyidir.

3.1.2 Problemlerin Formüle Edilmesi

Bir problem tanımı gerçek dünyanın kendisi değil, onun soyut matematiksel modelidir. "In(Arad)" gerçek bir yolculuktaki radyo programı, hava durumu, yakıt, manzara, yol kalitesi ve benzeri sayısız ayrıntıyı göz ardı eder. Bu ayrıntılar rota bulma hedefi için gerekli olmadığından durum tanımından çıkarılır.

Soyutlama (abstraction)

Bir gösterimden problem açısından gereksiz ayrıntıların çıkarılmasıdır. Soyutlama yalnızca durumlarda değil, eylemlerde de yapılır. Örneğin "Arad'dan Sibiu'ya git" eylemi binlerce küçük direksiyon, hızlanma ve fren hareketini tek bir üst düzey eylem olarak temsil eder.

İyi bir soyutlama iki özelliğe sahip olmalıdır: geçerli olmalı (soyut çözüm gerçek dünyada uygulanabilir bir ayrıntılı çözüme açılabilir) ve yararlı olmalı (soyut eylemlerin uygulanması asıl problemi yeniden

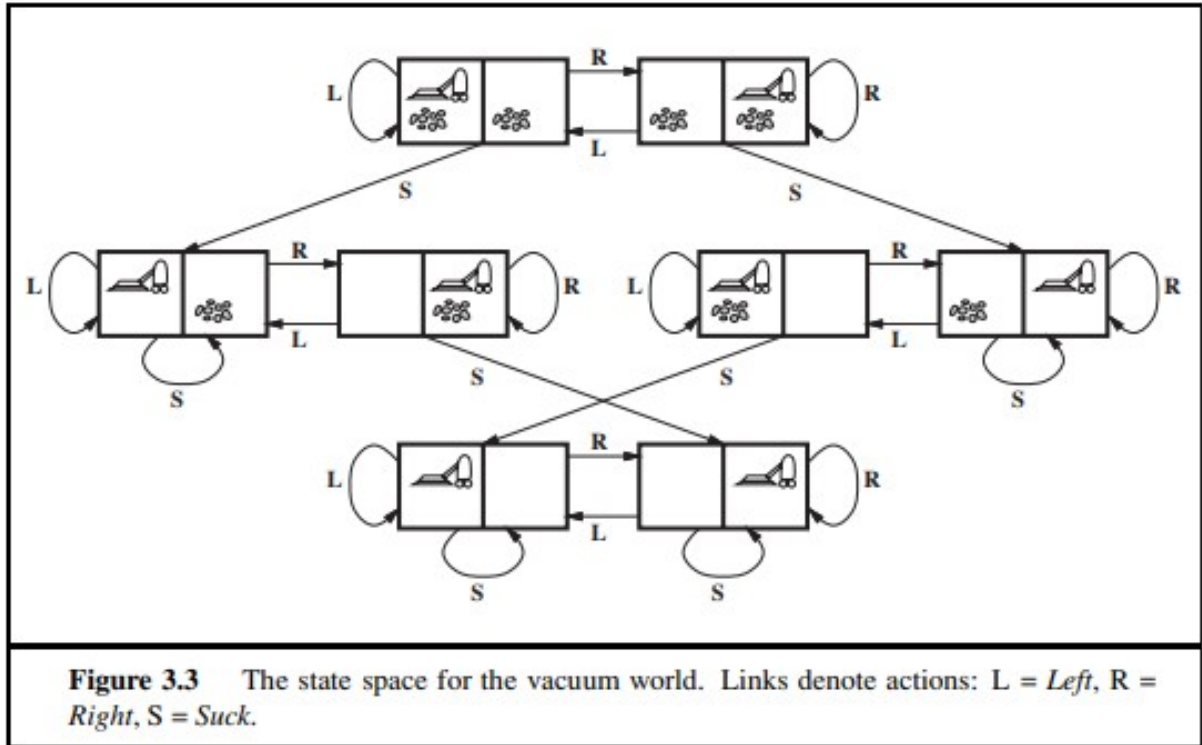
çözmek kadar zor olmamalıdır). Akıllı ajanların gerçek dünyanın ayrıntıları altında ezilmemesi için uygun soyutlamalar vazgeçilmezdir.

3.2 ÖRNEK PROBLEMLER

Problem çözme yaklaşımı çok farklı görev ortamlarına uygulanabilir. Kaynak iki sınıf ayırır: oyuncak problemler ve gerçek dünya problemleri. Oyuncak problemler kısa ve kesin biçimde tanımlanabilir; algoritmaları karşılaştırmak için kullanışlıdır. Gerçek dünya problemleri ise insanların gerçekten çözmek istediği, çoğu kez daha karmaşık ve tek bir standart tanımı olmayan problemlerdir.

3.2.1 Oyuncak Problemler

Vacuum World (Süpürge Dünyası)

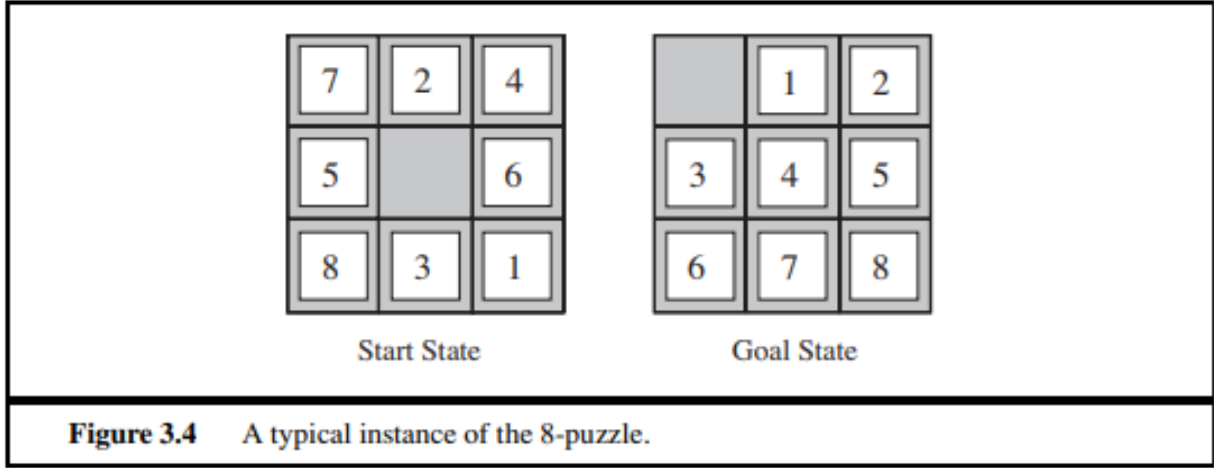


Şekil 3.3 - Süpürge dünyasının durum uzayı. Bağlantılar eylemleri gösterir: L = Sol, R = Sağ, S = Temizle (Suck).

Durum, ajanın konumu ve kirin bulunduğu karelerle belirlenir. İki konum vardır ve her kare temiz veya kirli olabilir. Bu nedenle toplam durum sayısı $2 \times 2^2 = 8$ 'dir. n konumlu daha büyük bir ortam için durum sayısı $n \times 2^n$ olur.

Bileşen	Vacuum World
Durumlar	Ajanın konumu + her karenin temiz/kirli durumu
Başlangıç	Herhangi bir durum
Eylemler	Left, Right, Suck
Geçiş modeli	Beklenen hareketler; duvara doğru hareket veya temiz karede Suck etkisizdir
Hedef testi	Bütün kareler temiz mi?
Yol maliyeti	Her eylem 1; maliyet = adım sayısı

8-Puzzle

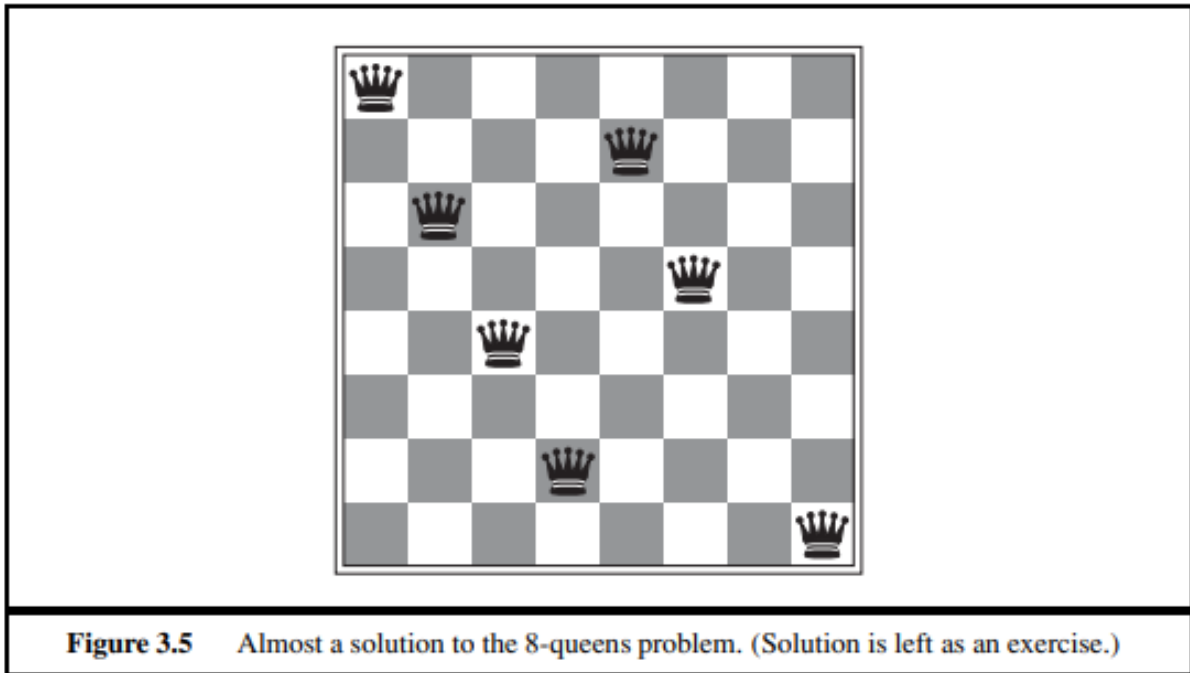


Şekil 3.4 - Tipik bir 8-Puzzle örneği: başlangıç ve hedef durumu.

8-Puzzle, 3x3 tahtada sekiz numaralı taş ve bir boşluktan oluşur. Boşluğa komşu bir taş boşluğa kaydırılabilir. Bir durum, sekiz taşın ve boşluğun dokuz karedeki yerlerini belirtir. Eylemler boşluğun Left, Right, Up ve Down hareketleri olarak tanımlanabilir; hangi hareketlerin mümkün olduğu boşluğun konumuna bağlıdır.

Her hareketin maliyeti 1 olduğundan en kısa çözüm en az hamleli çözümdür. Erişilebilir durum sayısı $9!/2 = 181.440$ 'tır. 15-Puzzle yaklaşık 1,3 trilyon durum, 24-Puzzle ise yaklaşık 10^{25} durum seviyesine ulaşır. Bu büyüme, arama uzaylarının kombinatorial patlamasını gösterir.

8-Queens (8 Vezir)



Şekil 3.5 - 8 vezir probleminde neredeyse çözüm olan bir yerleşim. Sağdaki vezir sol üstteki vezir tarafından çaprazdan tehdit edilir.

Amaç sekiz veziri, hiçbirinin aynı satır, sütun veya çapraz üzerinden başka bir vezire saldıramayacağı biçimde yerleştirmektir. İki ana formülasyon vardır: artımlı formülasyonda boş tahtaya vezirler teker teker eklenir; tam-durum formülasyonunda sekiz vezir baştan tahtadadır ve yerleri değiştirilir.

Kötü artımlı formülasyonda herhangi bir boş kareye vezir eklenebilir. Bu yaklaşım yaklaşık $64 \times 63 \times \dots \times 57 \approx 1,8 \times 10^{14}$ eylem dizisi doğurur. Daha iyi formülasyonda her yeni vezir soldaki ilk boş sütunda,

saldırı altında olmayan bir kareye konur. Durum uzayı yalnızca 2.057 duruma kadar küçülür. Böylece aynı fiziksel problem, daha iyi formüle edilerek dramatik biçimde kolaylaştırılır.

Knuth'un sonsuz durum uzayı örneği

Donald Knuth'un örneğinde başlangıç sayısı 4'tür ve eylemler faktöriyel, karekök ve floor işlemleridir. Amaç istenen pozitif tamsayıya ulaşmaktır. Hedef 5 gibi küçük bir sayı olsa bile çözüm yolunda 620.448.401.733.239.439.360.000 gibi çok büyük ara değerler oluşabilir. Bu nedenle durum uzayına sonlu bir üst sınır verilemez; durum uzayı sonsuzdur. Matematiksel ifade, devre, ispat ve program üretimi gibi görevlerde benzer sonsuz durum uzayları görülür.

3.2.2 Gerçek Dünya Problemleri

Rota bulma ve uçuş planlama

Rota bulma, belirli konumlar ve aralarındaki geçişlerle tanımlanır. Navigasyon sistemleri, bilgisayar ağlarında veri yönlendirme, askeri operasyon planlama ve havayolu seyahat planlama gibi birçok uygulaması vardır. Uçuş planlamada durum yalnızca havaalanını değil, zamanı ve önceki uçuşlardan kaynaklanan ücret/aktarma gibi geçmiş bilgilerini de içerebilir. Eylemler uygun uçuşları seçmek; geçiş modeli varış noktasını ve varış zamanını güncellemek; yol maliyeti ise ücret, bekleme, uçuş süresi, gümrük, koltuk kalitesi gibi çok sayıda faktörü birleştirmek zorundadır.

Touring Problem ve Traveling Salesperson Problem (TSP)

"Bütün şehirleri en az bir kez ziyaret et ve başladığın şehre dön" tipi problemlerde durum yalnızca mevcut konumu değil, ziyaret edilmiş şehirler kümesini de içermelidir. TSP'de her şehir tam bir kez ziyaret edilir ve en kısa tur aranır. TSP NP-hard bir problemdir; devre kartı delme makineleri ve fabrika içi stoklama makinelerinin hareketlerinin planlanması gibi uygulamalarda da kullanılır.

VLSI yerleşimi

VLSI yerleşim probleminde milyonlarca bileşen ve bağlantı, çip alanını ve gecikmeyi azaltıp üretim verimini artıracak biçimde yerleştirilir. Hücre yerleşimi, bileşen gruplarının üst üste gelmeyecek biçimde konumlandırılmasını; kanal yönlendirme ise hücreler arasındaki kablolar için yolların bulunmasını içerir.

Robot navigasyonu

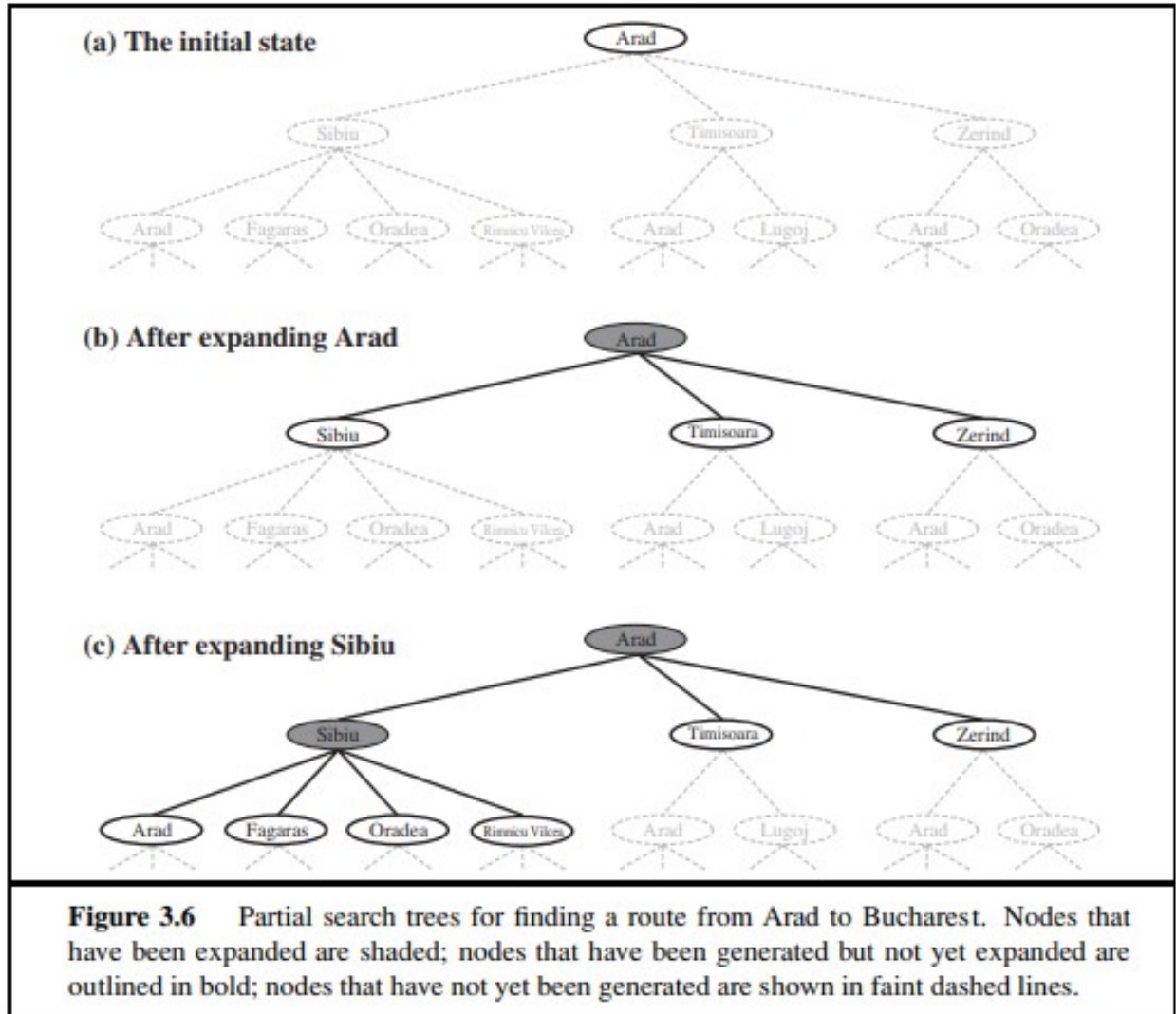
Robot navigasyonu rota bulmanın sürekli uzaya genelleştirilmiş biçimidir. Düz bir zeminde dairesel robot için durum yaklaşık (x,y) ile temsil edilebilir; kollar, bacaklar veya tekerlek açıları eklendiğinde arama uzayı çok boyutlu hale gelir. Gerçek robotlarda sensör ve motor hataları da dikkate alınmalıdır.

Otomatik montaj sıralaması ve protein tasarımı

Montaj sıralamasında parçaların hangi sırayla bir araya getirileceği aranır. Yanlış sıra, sonraki bir parçanın takılamamasına yol açabilir; üstelik bir adımın geometrik olarak mümkün olup olmadığını test etmek bile zor olabilir. Protein tasarımı ise amaç, belirli özelliklere sahip üç boyutlu bir proteine katlanacak amino asit dizisini bulmaktır.

3.3 ÇÖZÜM ARAMA

Çözüm bir eylem dizisi olduğuna göre arama algoritmaları farklı eylem dizilerini inceler. Başlangıç durumundan doğabilecek tüm eylem dizileri bir arama ağacı (search tree) oluşturur. Kök başlangıç durumudur; dallar eylemleri, düğümler ise durumları temsil eder.



Şekil 3.6 - Arad'dan Bükreş'e rota ararken arama ağacının ilk aşamaları. Genişletilen düğümler gri, üretilip henüz genişletilmeyen frontier düğümleri kalın çerçeveli, henüz üretilmeyenler soluk kesikli çizgilidir.

Arad önce hedef testinden geçirilir. Hedef değilse Arad genişletilir ve Sibiu, Timisoara, Zerind çocukları üretilir. Bunlardan biri seçilir; örneğin Sibiu genişletildiğinde Arad, Fagaras, Oradea ve Rimnicu Vilcea oluşur. Henüz çocuğu olmayan düğümler yaprak düğümlerdir. Herhangi bir anda genişletilmeye hazır tüm yaprakların kümesine frontier denir.

Frontier

Keşfedilmiş fakat henüz genişletilmemiş düğümlerin kümesidir. Arama stratejilerinin çoğu, frontier'dan sıradaki hangi düğümün seçileceğini belirleyerek birbirinden ayrılır.

```

TREE-SEARCH(problem)
  frontier <- initial state
  loop:
    if frontier empty: return failure
    node <- choose and remove a leaf from frontier
    if node is goal: return solution
    expand node and add children to frontier

```

Şekil 3.6'da Arad - Sibiu - Arad yolu aynı durumu yeniden üretir. Buna repeated state denir. Arad - Sibiu - Arad - Sibiu - ... gibi bir yol sonsuz döngüye dönüşebilir. Romanya durum uzayında yalnızca 20 şehir olmasına rağmen tekrarları içeren arama ağacı sonsuz olabilir.

Döngülü yollar, redundant path (gereksiz/yinelenen yol) kavramının özel durumudur. Örneğin Arad - Sibiu 140 km iken Arad - Zerind - Oradea - Sibiu 297 km'dir; ikisi de aynı duruma ulaşır ama ikinci yol daha

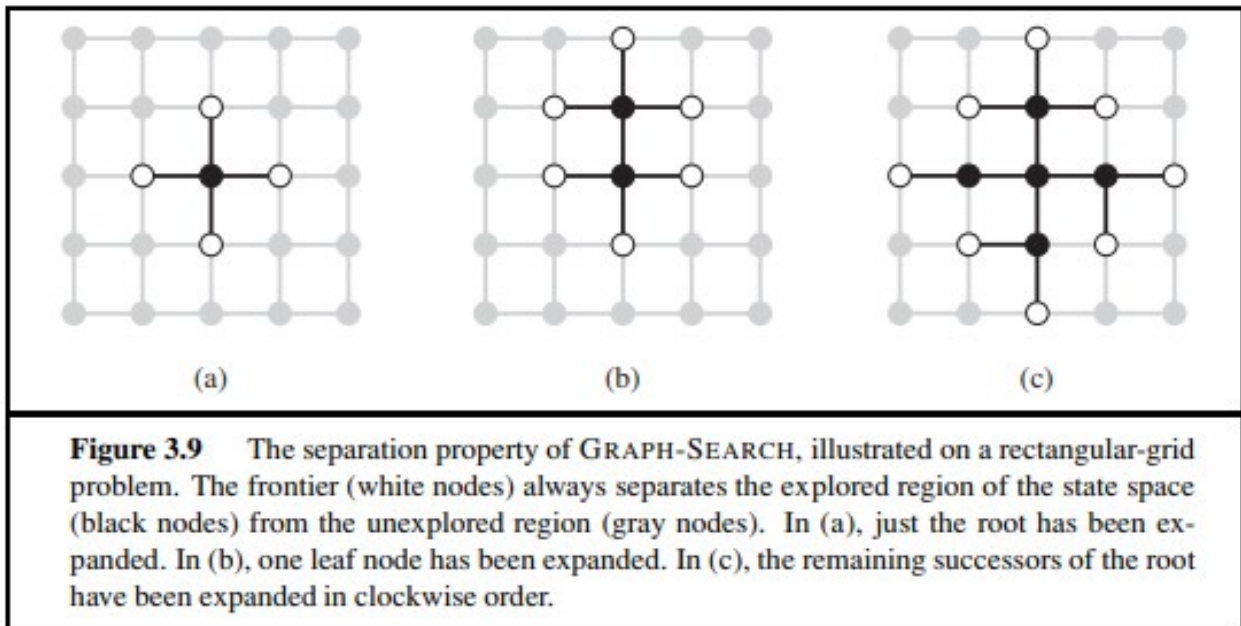
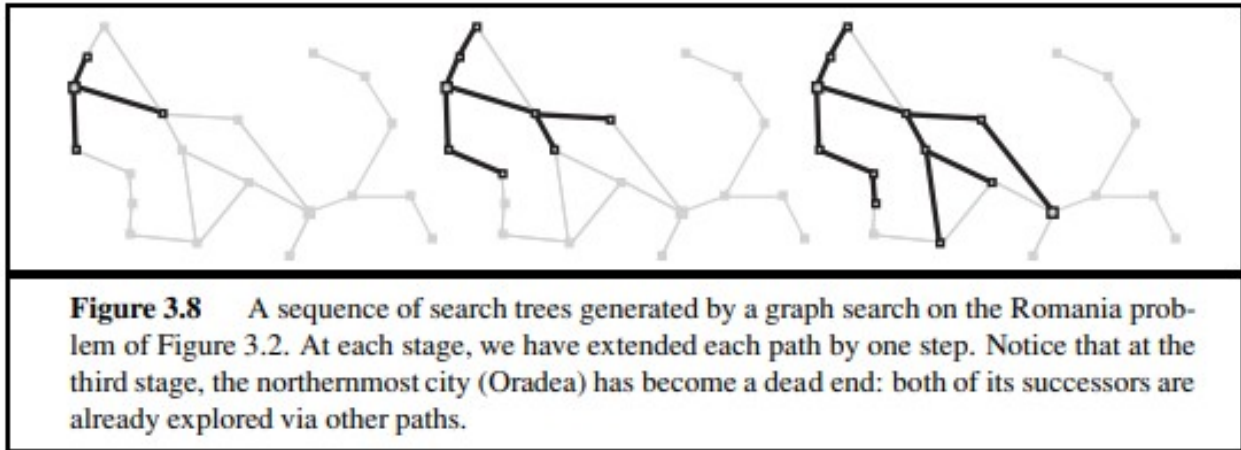
pahalıdır. Aynı duruma giden daha kötü yolları tekrar tekrar genişletmek büyük performans kaybına neden olur.

```

GRAPH-SEARCH(problem)
  frontier <- initial state
  explored <- empty set
  loop:
    if frontier empty: return failure
    node <- choose and remove a leaf from frontier
    if node is goal: return solution
    add node.state to explored
    expand node
    add child only if its state is not in frontier or explored

```

Graph Search, daha önce genişletilen durumları explored set içinde hatırlar. Yeni üretilen bir durum frontier veya explored içinde zaten varsa atılabilir. Böylece tekrarlar ve döngüler büyük ölçüde önlenir.



Şekil 3.8 ve 3.9 - Üstte Romanya problemi üzerinde Graph Search ilerleyişi; altta frontier'ın explored ve unexplored bölgeleri ayırma özelliği. Siyah düğümler explored, beyaz düğümler frontier, gri düğümler unexplored bölgedir.

3.3.1 Arama Algoritmalarının Altyapısı

Arama ağacını takip etmek için her düğümden dört temel bilgi tutulur: n.STATE, n.PARENT, n.ACTION ve n.PATH-COST. Yol maliyeti geleneksel olarak $g(n)$ ile gösterilir.

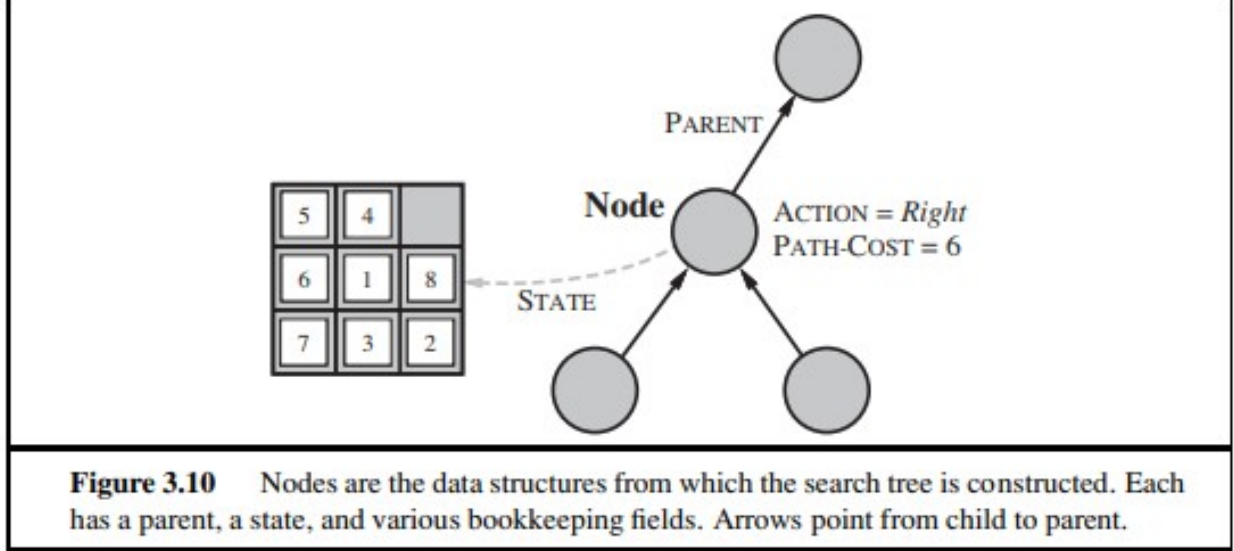
Alan	Anlamı
STATE	Düğümün temsil ettiği dünya durumu
PARENT	Bu düğümü üreten ebeveyn düğüm

ACTION	Ebeveynden bu düğüme geçişte kullanılan eylem
PATH-COST = g(n)	Başlangıçtan bu düğüme kadar gerçek yol maliyeti

```

CHILD-NODE(problem, parent, action)
  STATE = RESULT(parent.STATE, action)
  PARENT = parent
  ACTION = action
  PATH-COST = parent.PATH-COST + STEP-COST(parent.STATE, action)

```



Şekil 3.10 - Search tree düğümünün veri yapısı. Her düğüm state, parent, action ve path-cost gibi alanları taşır; oklar child'dan parent'a doğrudur.

Düğüm ile durum aynı şey değildir. Durum dünyanın bir konfigürasyonudur; düğüm ise arama algoritmasının bookkeeping veri yapısıdır. Aynı state farklı yollarla üretildiğinde farklı node'larda bulunabilir; parent ve path-cost değerleri farklı olabilir.

Frontier, sıradaki düğümün seçilmesini kolaylaştıracak bir kuyruk yapısında saklanır. Üç yaygın tür vardır: FIFO (ilk giren ilk çıkar; BFS), LIFO/stack (son giren ilk çıkar; DFS) ve priority queue (öncelik fonksiyonuna göre en uygun eleman çıkarılır; UCS, Greedy, A*). Explored set ise tekrar durum kontrolünü hızlı yapmak için çoğunlukla hash table ile uygulanır.

Canonical form

Mantıksal olarak aynı durumların aynı veri gösterimine dönüştürülmesidir. Örneğin ziyaret edilen şehirler kümesi {Bucharest, Urziceni, Vaslui} ile {Vaslui, Bucharest, Urziceni} aynı durum sayılmalıdır.

3.3.2 Problem Çözme Performansının Ölçülmesi

Ölçüt	Soru
Completeness (Tamlık)	Çözüm varsa algoritma onu bulmayı garanti ediyor mu?
Optimality (Optimallik)	Bulunan çözüm mümkün olan en iyi çözüm mü?
Time Complexity	Çözümü bulmak için ne kadar hesaplama gerekir?
Space Complexity	Arama sırasında ne kadar bellek gerekir?

AI aramalarında durum grafiği çoğu zaman açıkça verilmez; başlangıç durumu, eylemler ve geçiş modeliyle örtük biçimde tanımlanır. Bu nedenle karmaşıklık genellikle üç parametreyle ifade edilir:

Sembol	Anlam
b	Branching factor: bir düğümün maksimum ardıl sayısı

d	En sığ hedef düğümünün derinliği
m	Durum uzayındaki herhangi bir yolun maksimum uzunluğu

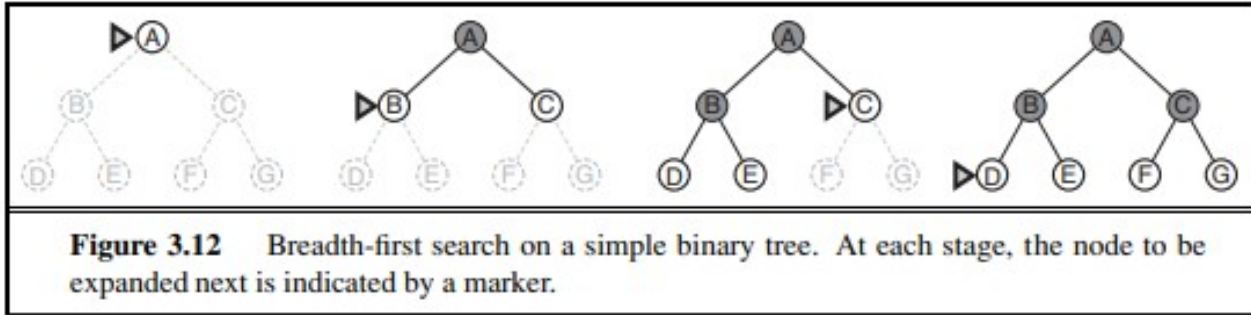
Zaman çoğunlukla üretilen düğüm sayısı, bellek ise aynı anda saklanan maksimum düğüm sayısı ile ölçülür. Search cost algoritmanın hesaplama maliyetidir; solution/path cost bulunan çözüm yolunun maliyetidir. Total cost, bazı uygulamalarda ikisini ortak bir ölçekte birleştirebilir.

3.4 BİLGİSİZ ARAMA STRATEJİLERİ

Bilgisiz (uninformed veya blind) arama, problem tanımının ötesinde hangi hedef olmayan durumun daha umut verici olduğuna ilişkin ek bilgi kullanmaz. Algoritma ardılları üretebilir ve hedefi hedef olmayandan ayırabilir. Bilgisiz arama stratejileri temel olarak düğümlerin hangi sırayla genişletildiğiyle birbirinden ayrılır.

3.4.1 Genişlik Öncelikli Arama (Breadth-First Search - BFS)

BFS önce kökü, sonra kökün tüm çocuklarını, ardından bir sonraki seviyenin tüm düğümlerini genişletir. Başka bir deyişle daha derin seviyeye geçmeden önce mevcut derinlikteki tüm düğümler tamamlanır. Frontier FIFO kuyruğudur; en eski ve dolayısıyla en sığ düğüm önce çıkar.



Şekil 3.12 - Basit ikili ağaçta Breadth-First Search. Her aşamada küçük işaret sırada genişletilecek düğümü gösterir.

```

BREADTH-FIRST-SEARCH(problem)
  node ← initial state, path-cost = 0
  if node is goal: return solution
  frontier ← FIFO(node)
  explored ← {}
  loop:
    if frontier empty: return failure
    node ← POP(frontier)
    add node.state to explored
    for each action:
      child ← CHILD-NODE(...)
      if child.state not in explored/frontier:
        if child is goal: return solution
        INSERT(child, frontier)

```

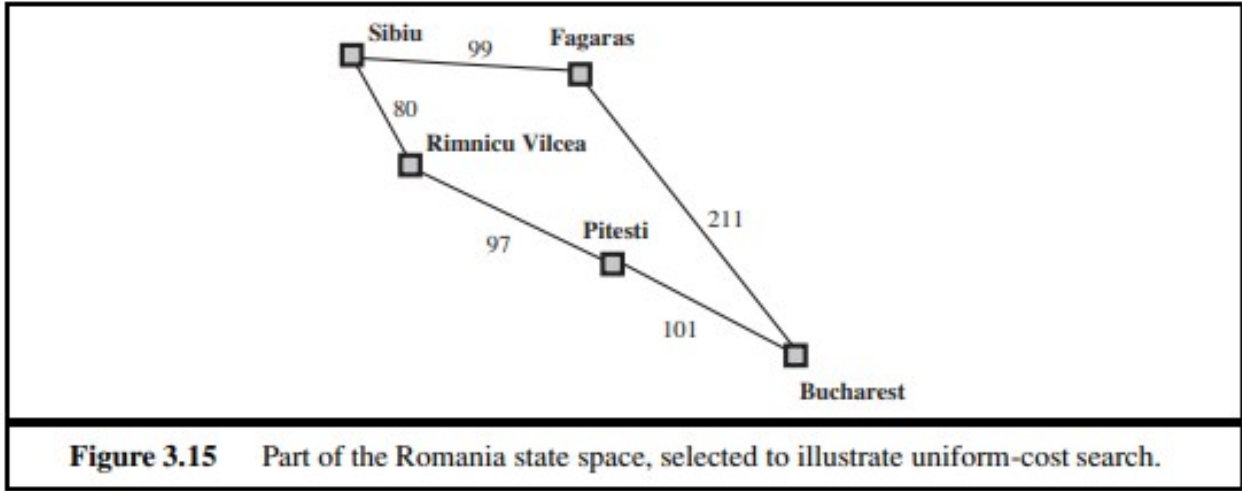
BFS, branching factor sonlu ve en sığ hedef sonlu derinlikteyse complete'tir. Yol maliyeti derinlikle azalmıyorsa - özellikle bütün adım maliyetleri eşitse - optimaldir. Çözüm derinliği d ve branching factor b olduğunda zaman ve bellek karmaşıklığı $O(b^d)$ düzeyindedir. En büyük dezavantaj, frontier'ın büyüklüğü nedeniyle bellek tüketimidir.

BFS özeti

Seviye seviye gider. Veri yapısı FIFO'dur. Complete: evet (b sonluysa). Optimal: eşit adım maliyetlerinde evet. Zaman: $O(b^d)$. Bellek: $O(b^d)$.

3.4.2 Tekdüzen Maliyetli Arama (Uniform-Cost Search - UCS)

UCS en sık düğümü değil, $g(n)$ değeri yani başlangıçtan o düğüme kadarki toplam yol maliyeti en düşük olan düğümü genişletir. Frontier, $g(n)$ 'e göre sıralanan priority queue'dur. Hedef testi düğüm üretilirken değil, düğüm en düşük maliyetli eleman olarak frontier'dan çıkarıldığında yapılır; çünkü ilk üretilen hedef optimal olmayabilir.



Şekil 3.15 - Uniform-Cost Search'ü açıklamak için seçilmiş Romanya durum uzayı bölümü.

Sibiu'dan Bükreş'e örneğinde Rimnicu Vilcea 80, Fagaras 99 maliyetle üretilir. Önce Rimnicu seçilir; Pitesti'nin maliyeti $80+97=177$ olur. Sonra Fagaras 99 ile seçilir ve 310 maliyetli Bucharest üretilir. Ancak UCS durmaz; Pitesti 177 daha ucuzdur. Pitesti genişletildiğinde Bucharest'e $80+97+101=278$ maliyetli daha iyi yol bulunur ve 310'luk yol değiştirilir.

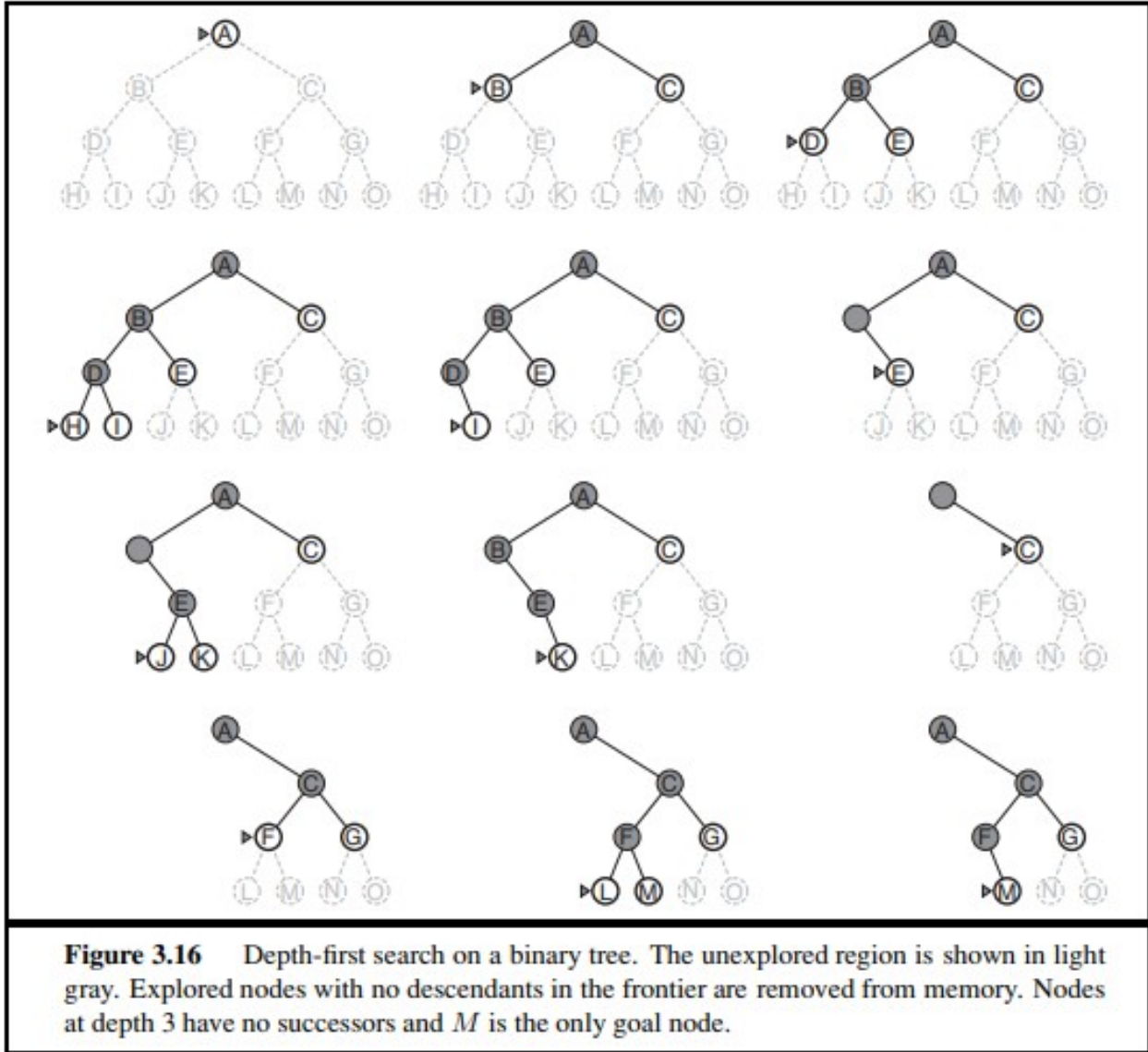
Adım maliyetleri sıfırdan büyük bir alt sınıra sahipse UCS complete ve genel olarak optimaldir. Zaman/bellek karmaşıklığı d ile değil optimal çözüm maliyeti C^* ve minimum adım maliyeti ϵ ile ifade edilir; kötü durumda $O(b^{(1+\text{floor}(C^*/\epsilon))})$ seviyesine çıkabilir.

UCS özeti

En düşük $g(n)$ değerini seçer. Veri yapısı priority queue'dur. Eşit adım maliyetlerinde BFS'ye benzer. Optimal çözüm için maliyetlerin negatif olmaması ve tamlık için pozitif alt sınır koşulu önemlidir.

3.4.3 Derinlik Öncelikli Arama (Depth-First Search - DFS)

DFS frontier'daki en derin düğümü genişletir. LIFO/stack kullanılır; en son üretilen düğüm önce seçilir. Arama bir dalda mümkün olduğunca aşağı gider; yaprak veya çıkmaz noktada geri dönerek henüz keşfedilmemiş kardeşleri dener.



Şekil 3.16 - İkili ağaçta Depth-First Search. Açık gri bölge henüz keşfedilmemiştir; frontier'da ardılı kalmayan explored düğümler bellekten çıkarılır. *M* tek hedef düğümüdür.

DFS'nin özellikleri Tree Search ve Graph Search sürümüne göre değişir. Sonlu durum uzayında Graph Search tekrarları engellediğinden complete olabilir. Tree Search ise Arad - Sibiu - Arad döngüsüne girerek sonsuza kadar devam edebilir; bu nedenle complete değildir. Her iki sürüm de genel olarak optimal değildir.

Tree Search DFS'nin zaman karmaşıklığı $O(b^m)$ olabilir. Büyük avantajı bellektir: yalnızca mevcut kök-yaprak yolu ve genişletilmemiş kardeşler tutulduğundan $O(bm)$ bellek yeterlidir. Backtracking search yalnızca bir ardılı bir seferde üretirek belleği $O(m)$ 'e kadar indirebilir.

3.4.4 Derinlik Sınırlı Arama (Depth-Limited Search - DLS)

DFS'nin sonsuz derinliğe gitme sorunu önceden belirlenen bir derinlik sınırı l ile hafifletilebilir. Derinliği l olan düğümler çocukları yokmuş gibi ele alınır. Böylece sonsuz yol problemi çözülür; fakat $l < d$ ise en sık hedef limitin ötesinde kalır ve algoritma tamamlayıcı olmaz. $l > d$ olduğunda optimality de garanti değildir.

```
DEPTH-LIMITED-SEARCH(problem, limit)
  return RECURSIVE-DLS(initial-node, problem, limit)
```

```
RECURSIVE-DLS(node, problem, limit)
  if node is goal: return solution
  else if limit = 0: return cutoff
  else:
    cutoff_occurred <- false
```

for each action:

```

child <- CHILD-NODE(...)
result <- RECURSIVE-DLS(child, problem, limit-1)
if result = cutoff: cutoff_occurred <- true
else if result != failure: return result
if cutoff_occurred: return cutoff else return failure

```

DLS iki farklı olumsuz sonuç verir: failure “bu arama alanında çözüm yok” demektir; cutoff ise “limit yüzünden daha derine bakamadım, çözüm olabilir” anlamına gelir. Romanya’da 20 şehir olduğu için 19 kaba bir limit olabilir; haritanın çapı incelendiğinde herhangi iki şehir arasında en fazla 9 adım gerektiği görülür ve $l=9$ daha iyi bir sınırdır.

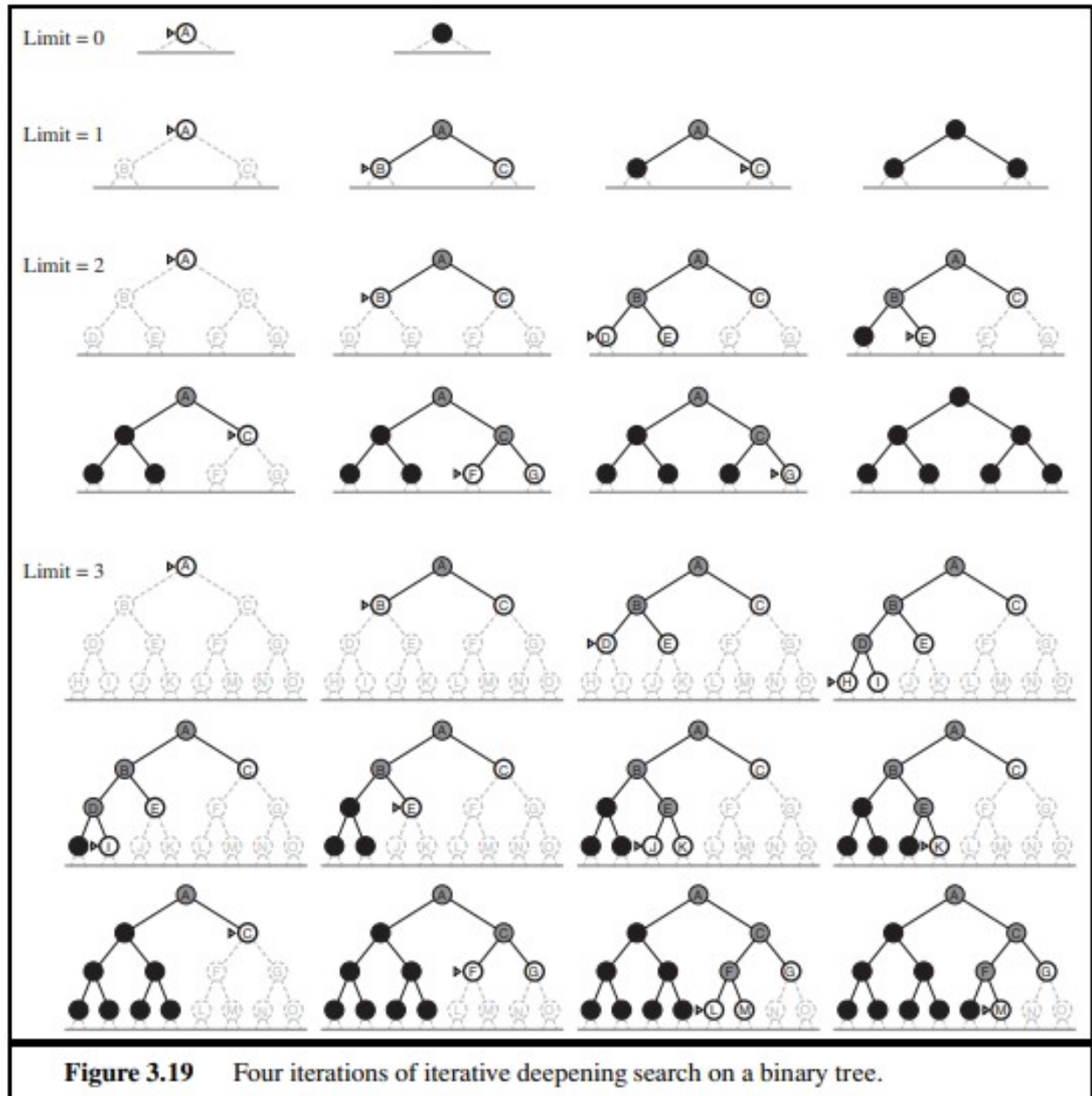
3.4.5 Yinelemeli Derinleşen Arama (Iterative Deepening Search - IDS)

Çözüm derinliği bilinmiyorsa depth limit 0, 1, 2, ... biçiminde adım adım artırılır. Her iterasyonda Depth-Limited Search yeniden çalıştırılır ve limit en sığ hedef derinliği d’ye ulaştığında çözüm bulunur.

```

ITERATIVE-DEEPENING-SEARCH(problem)
  for depth = 0 to infinity:
    result <- DEPTH-LIMITED-SEARCH(problem, depth)
    if result != cutoff: return result

```

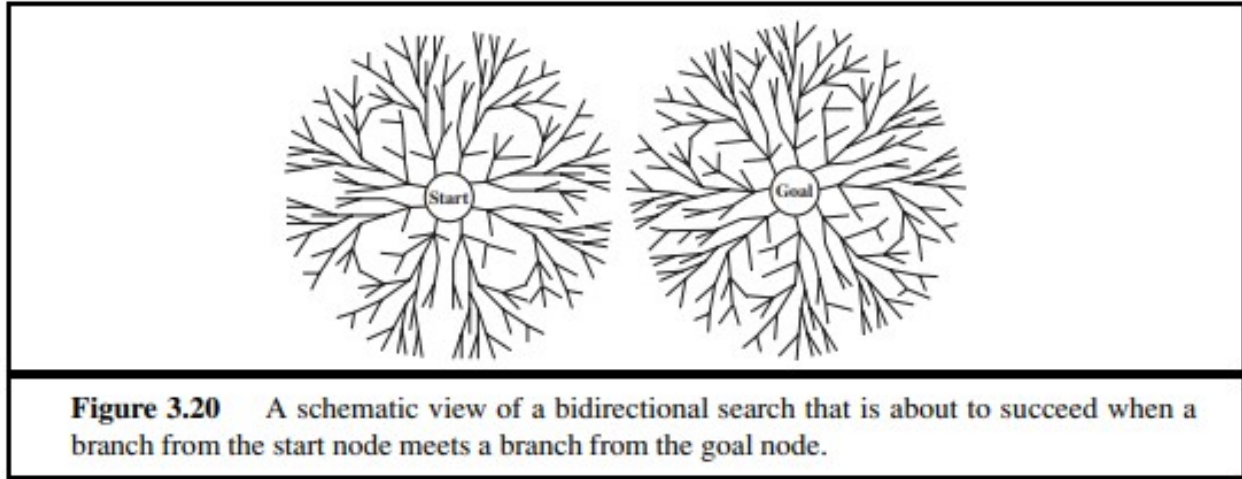


Şekil 3.19 - İkili ağaçta iterative deepening'in dört iterasyonu: limit 0, 1, 2 ve 3.

Üst seviyeler tekrar tekrar üretilse de düğümlerin büyük çoğunluğu en alt seviyede bulunduğundan ek maliyet düşündüğümüz kadar büyük değildir. $b=10$, $d=5$ için kaynakta IDS 123.450, BFS ise 111.110 düğüm üretmektedir. IDS zaman açısından $O(b^d)$, bellek açısından $O(bd)$ düzeyindedir. b sonluysa complete, eşit adım maliyetlerinde optimaldir ve büyük arama uzaylarında çözüm derinliği bilinmediğinde güçlü bir bilgisiz arama seçeneğidir.

3.4.6 Çift Yönlü Arama (Bidirectional Search)

Çift yönlü aramada iki arama eşzamanlı yürütülür: biri başlangıçtan ileriye, diğeri hedeften geriye. Amaç iki frontier'ın ortada kesişmesidir. Standart BFS'nin yaklaşık b^d büyümesine karşılık iki tarafta $d/2$ derinliğe gidildiğinde toplam maliyet yaklaşık $O(b^{(d/2)})$ olur.



Şekil 3.20 - Başlangıçtan ve hedeften büyüyen iki arama ağacının ortada buluşmasını gösteren çift yönlü arama şeması.

Örneğin $d=6$, $b=10$ için standart BFS yaklaşık 1.111.110 düğüm üretirken iki yöndeki BFS yaklaşık 2.220 düğümde buluşabilir. Ancak geriye doğru arama için bir durumun predecessors (öncülleri) üretilebilmelidir. Eylemler tersinir olduğunda bu kolaydır; soyut hedef tanımlarında veya ters eylemi olmayan problemlerde zor olabilir. Ayrıca frontier kesişmesi ilk bulunduğu anda optimality'yi garanti etmek için ek kontrol gerekebilir.

3.4.7 Bilgisiz Arama Stratejilerinin Karşılaştırılması

Yöntem	Complete?	Optimal?	Zaman	Bellek
BFS	Evet, b sonluysa	Eşit adım maliyetlerinde evet	$O(b^d)$	$O(b^d)$
UCS	Evet, $\text{step cost} \geq \epsilon > 0$	Evet	$O(b^{(1+\text{floor}(C^*/\epsilon))})$	Aynı merteye
DFS Tree Search	Hayır	Hayır	$O(b^m)$	$O(bm)$
Depth-Limited	Genelde hayır	Hayır	$O(b^l)$	$O(bl)$
Iterative Deepening	Evet, b sonluysa	Eşit adım maliyetlerinde evet	$O(b^d)$	$O(bd)$
Bidirectional (BFS/BFS)	Uygunsa evet	Koşullara bağlı	$O(b^{(d/2)})$	$O(b^{(d/2)})$

Öğrenci için kısa hatırlatma: BFS = en sığ; UCS = en ucuz $g(n)$; DFS = en derin; DLS = en derin ama limite kadar; IDS = limiti artırarak DFS; Bidirectional = iki taraftan ara.

3.5 BİLGİLİ (SEZGİSEL) ARAMA STRATEJİLERİ

Bilgili arama, problem tanımına ek olarak probleme özgü bilgi kullanarak çözümü daha verimli bulmaya çalışır. Genel yaklaşım best-first search'tür. Düğümler bir değerlendirme fonksiyonu $f(n)$ ile puanlanır ve en düşük değere sahip düğüm önce genişletilir. f 'nin seçimi arama stratejisini belirler.

Birçok best-first yöntem, $h(n)$ sezgisel fonksiyonunu kullanır:

$$h(n) = n \text{ durumundan hedefe giden en ucuz yolun tahmini maliyeti}$$

$h(n)$ düğümü girdi olarak alsa da $g(n)$ 'den farklı olarak yalnızca düğümdeki state'e bağlıdır. Hedef düğümünde $h(n)=0$ olmalıdır.

3.5.1 Greedy Best-First Search

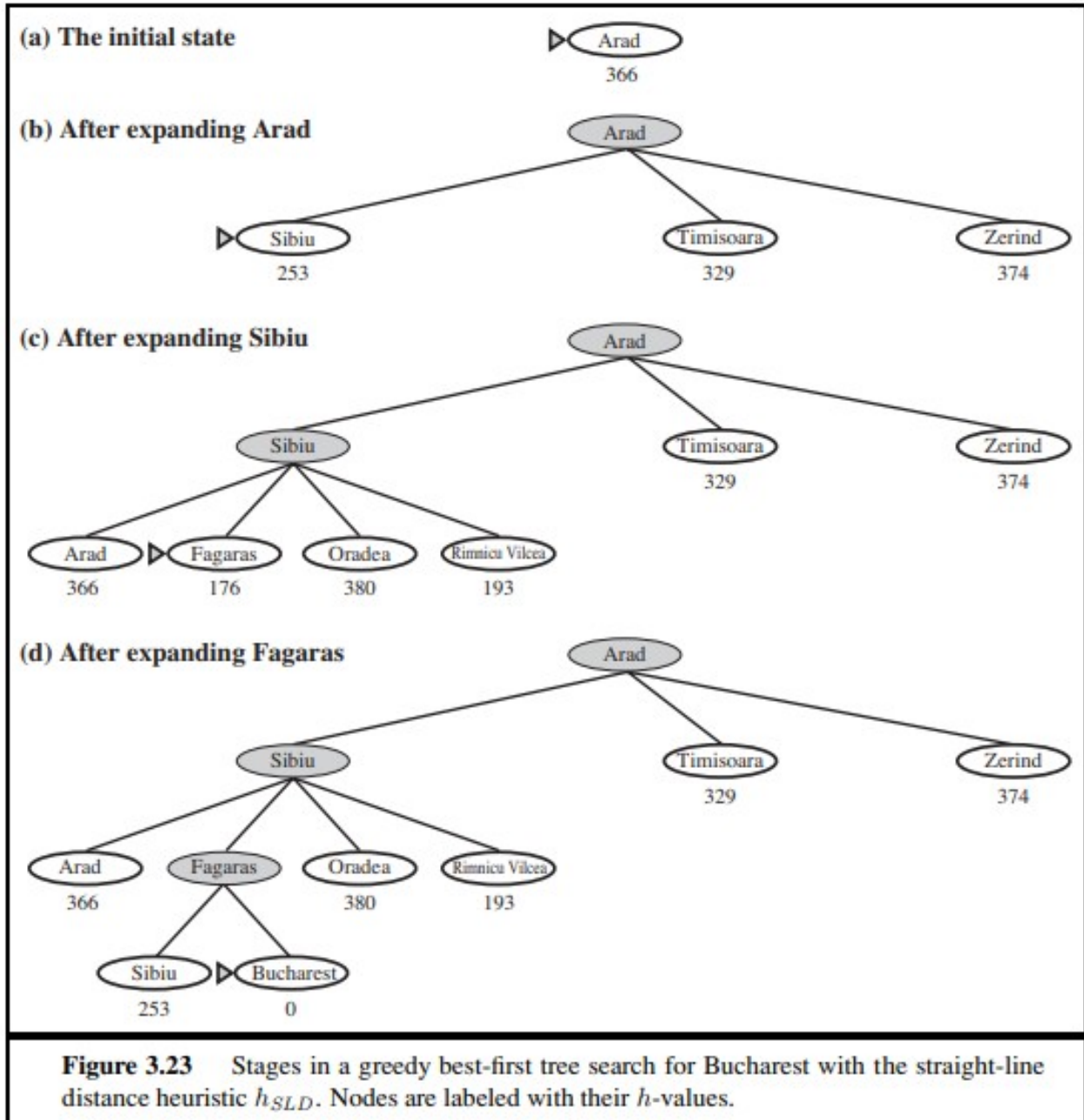
Greedy Best-First Search hedefe en yakın görünen düğümü genişletmeye çalışır. Değerlendirme fonksiyonu yalnızca heuristic'tir:

$$f(n) = h(n)$$

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Figure 3.22 Values of h_{SLD} —straight-line distances to Bucharest.

Şekil 3.22 - Bükreş'e kuş uçuşu uzaklıklar h_{SLD} . Örneğin $h(\text{Arad})=366$, $h(\text{Sibiu})=253$, $h(\text{Pitesti})=100$ ve $h(\text{Bucharest})=0$.



Şekil 3.23 - h_{SLD} kullanılarak Arad'dan Bükreş'e Greedy Best-First Tree Search aşamaları. Düğümler h değerleriyle etiketlenmiştir.

Arad'ın çocukları arasında Sibiu'nun h değeri 253 ile Timisoara 329 ve Zerind 374'ten küçüktür, bu yüzden Sibiu seçilir. Sibiu genişletildiğinde Fagaras 176, Rimnicu Vilcea 193'tür; Greedy Fagaras'ı seçer. Fagaras Bucharest'i üretir ve hedef bulunur. Ancak bulunan yol Arad - Sibiu - Fagaras - Bucharest = 450 km'dir; Rimnicu Vilcea - Pitesti üzerinden optimal yol 418 km'dir. Greedy hızlı olabilir ama optimal değildir.

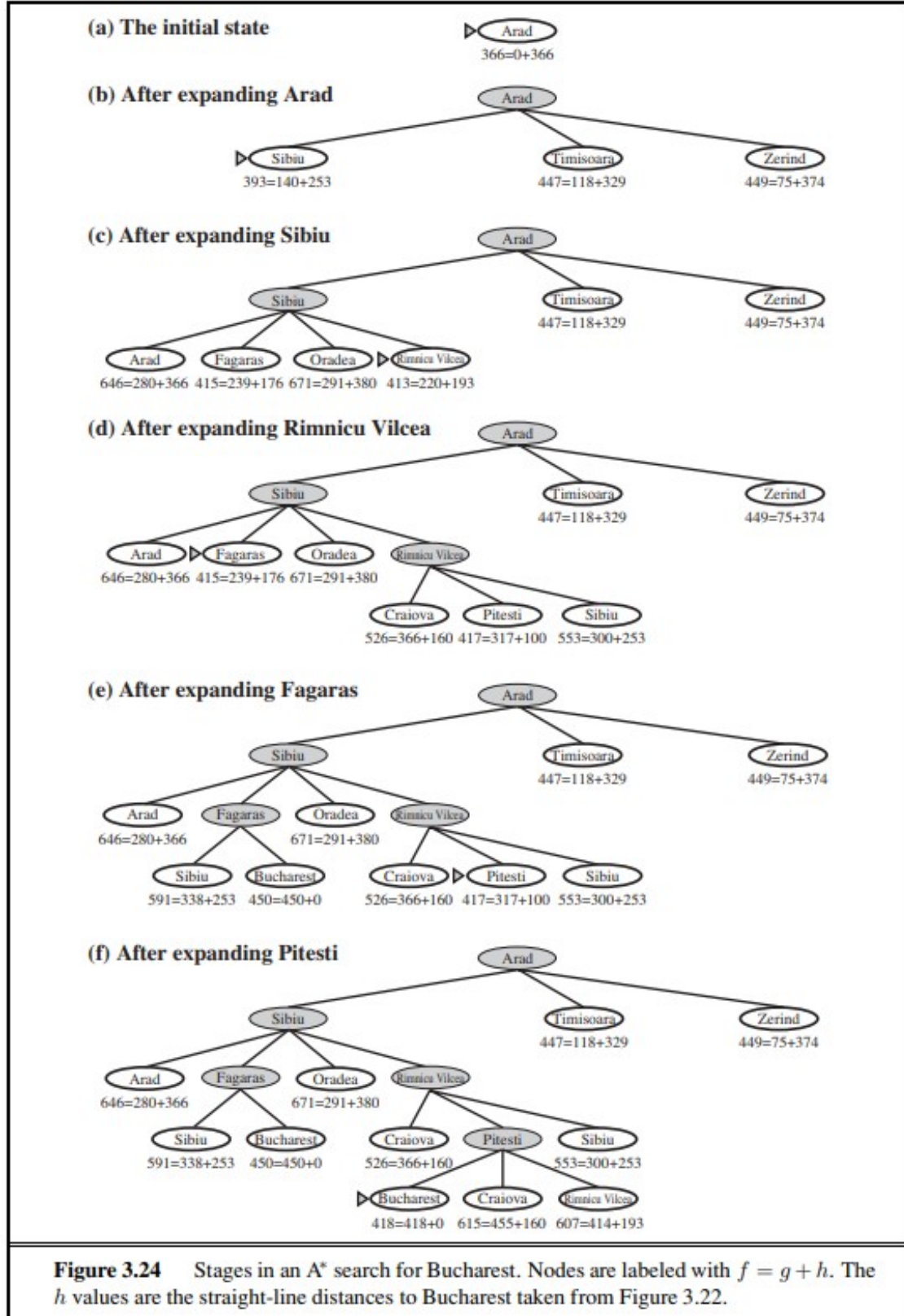
Tree Search Greedy sonlu bir durumda bile döngüye girebilir; Graph Search sonlu state space'lerde tekrarları engelleyerek complete olabilir. Kötü durumda zaman ve bellek $O(b^m)$ olabilir; iyi heuristic performansı büyük ölçüde iyileştirebilir.

3.5.2 A* Araması: Tahmini Toplam Çözüm Maliyetini En Aza İndirmek

A* geçmiş maliyet ile kalan tahmini maliyeti birleştirir:

$$f(n) = g(n) + h(n)$$

$g(n)$, başlangıçtan n 'e gerçek maliyettir. $h(n)$, n 'den hedefe kalan en ucuz maliyetin tahminidir. Dolayısıyla $f(n)$, n üzerinden geçen en ucuz çözümün tahmini toplam maliyetidir.



Şekil 3.24 - Arad'dan Bükreş'e A* aramasının aşamaları. Düğümler $f = g + h$ ile etiketlenmiştir; h değerleri Şekil 3.22'deki kuş uçuşu uzaklıklardır.

A* örneği adım adım: Arad için $f=0+366=366$. Arad genişletilince Sibiu $140+253=393$, Timisoara $118+329=447$, Zerind $75+374=449$ olur; Sibiu seçilir. Sibiu'dan Rimnicu Vilcea $220+193=413$, Fagaras $239+176=415$, Oradea $291+380=671$ çıkar; Rimnicu seçilir. Rimnicu'dan Pitesti $317+100=417$ olur. Sonra Fagaras 415 seçilir ve Bucharest $450+0=450$ üretilir; fakat Pitesti 417 daha umut vericidir. Pitesti genişletildiğinde Bucharest $418+0=418$ bulunur ve optimal çözüm elde edilir.

g, h, f

$g(n)$: şimdiye kadar gerçekten ödediğimiz maliyet. $h(n)$: hedefe kalan tahmini maliyet. $f(n)$: n üzerinden gidersek çözümün tahmini toplam maliyeti. UCS $f=g$ kullanır; Greedy $f=h$; A* $f=g+h$ kullanır.

Admissibility (Kabul edilebilirlik)

Bir heuristic, hedefe kalan gerçek minimum maliyeti asla olduğundan büyük tahmin etmiyorsa admissible'dır:

$$h(n) \leq h^*(n)$$

Kuş uçuşu mesafe h_SLD admissible'dır; iki nokta arasındaki düz çizgi gerçek yol mesafesinden uzun olamaz. Admissible heuristic'ler bu anlamda iyimser tahminlerdir.

Consistency (Tutarlılık / Monotonicity)

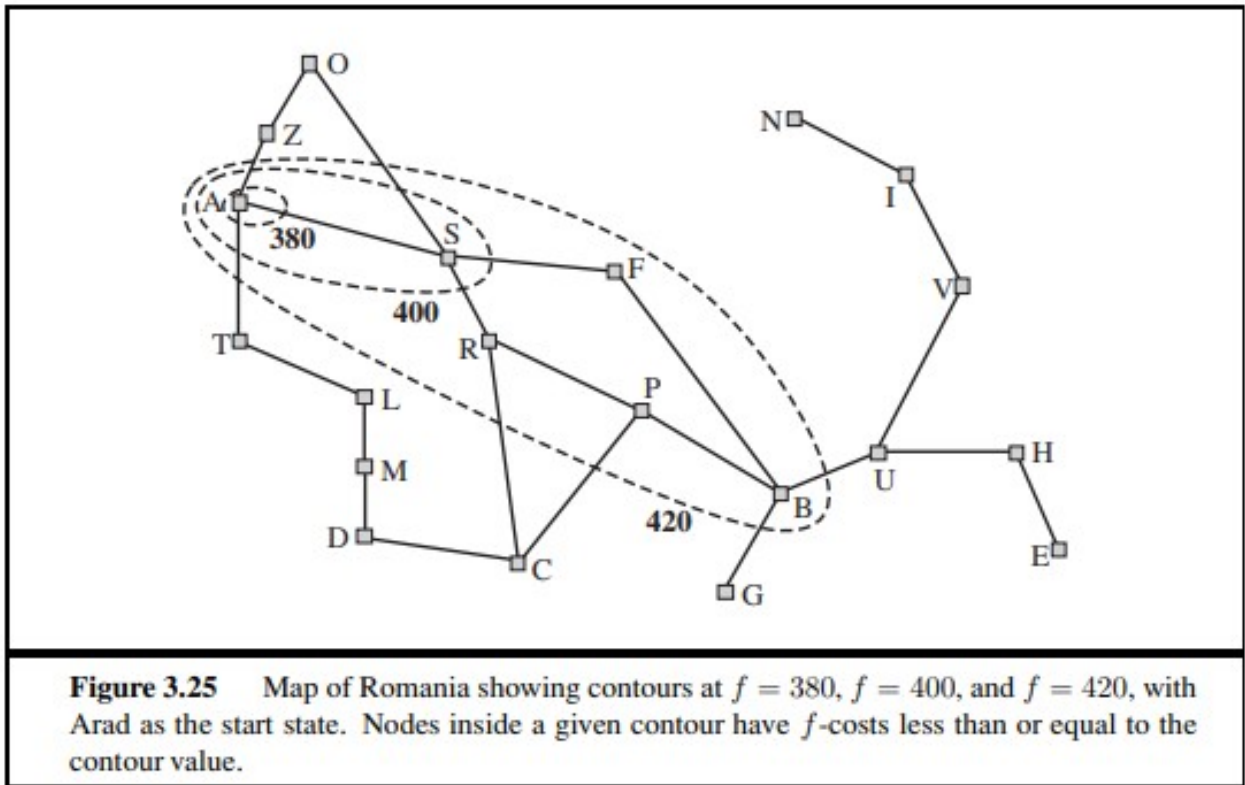
Graph Search A* için daha güçlü bir koşul consistency'dir. Her n düğümü ve ardılı n' için:

$$h(n) \leq c(n,a,n') + h(n')$$

Bu, üçgen eşitsizliğinin arama problemindeki karşılığıdır. Her consistent heuristic admissible'dır. Consistency varsa yol boyunca f değerleri azalmaz: $f(\text{child}) \geq f(\text{parent})$.

A* optimality ve contour yorumu

Tree Search A* admissible h ile; Graph Search A* ise consistent h ile optimaldir. Consistency altında A* düğümleri azalmayan f sırasıyla genişletir. Hedef düğümünde $h=0$ olduğundan ilk genişletilen hedefin f değeri gerçek çözüm maliyetidir ve optimal çözümü verir.



Şekil 3.25 - Arad başlangıçlı Romanya haritasında $f=380$, $f=400$ ve $f=420$ konturları. Bir konturun içindeki düğümlerin f maliyetleri o değere eşit veya küçüktür.

A* en küçük f değerini genişlettiği için arama, giderek büyüyen f -maliyet bantları halinde yayılır. $h=0$ olursa A* UCS'ye dönüşür. Heuristic daha doğru oldukça bantlar hedefe doğru uzar ve optimal yol çevresinde daralır. Optimal çözüm maliyeti C^* ise A* tüm $f < C^*$ düğümlerini genişletir, $f = C^*$ üzerindeki bazı düğümleri genişletebilir ve admissible/consistent koşullarıyla $f > C^*$ düğümlerini genişletmez. Bu gereksiz alt ağaçları incelemeyi eleme işlemine pruning (budama) denir.

A* aynı heuristic bilgiyi kullanan kökten-yol-genişleten optimal algoritmalar arasında optimally efficient kabul edilir. Ancak state sayısı çoğu problemde hâlâ üstel büyür. Özellikle Graph Search A* üretilen düğümleri bellekte tuttuğu için pratikte çoğu kez zamandan önce bellek tüketimi sınırına ulaşır.

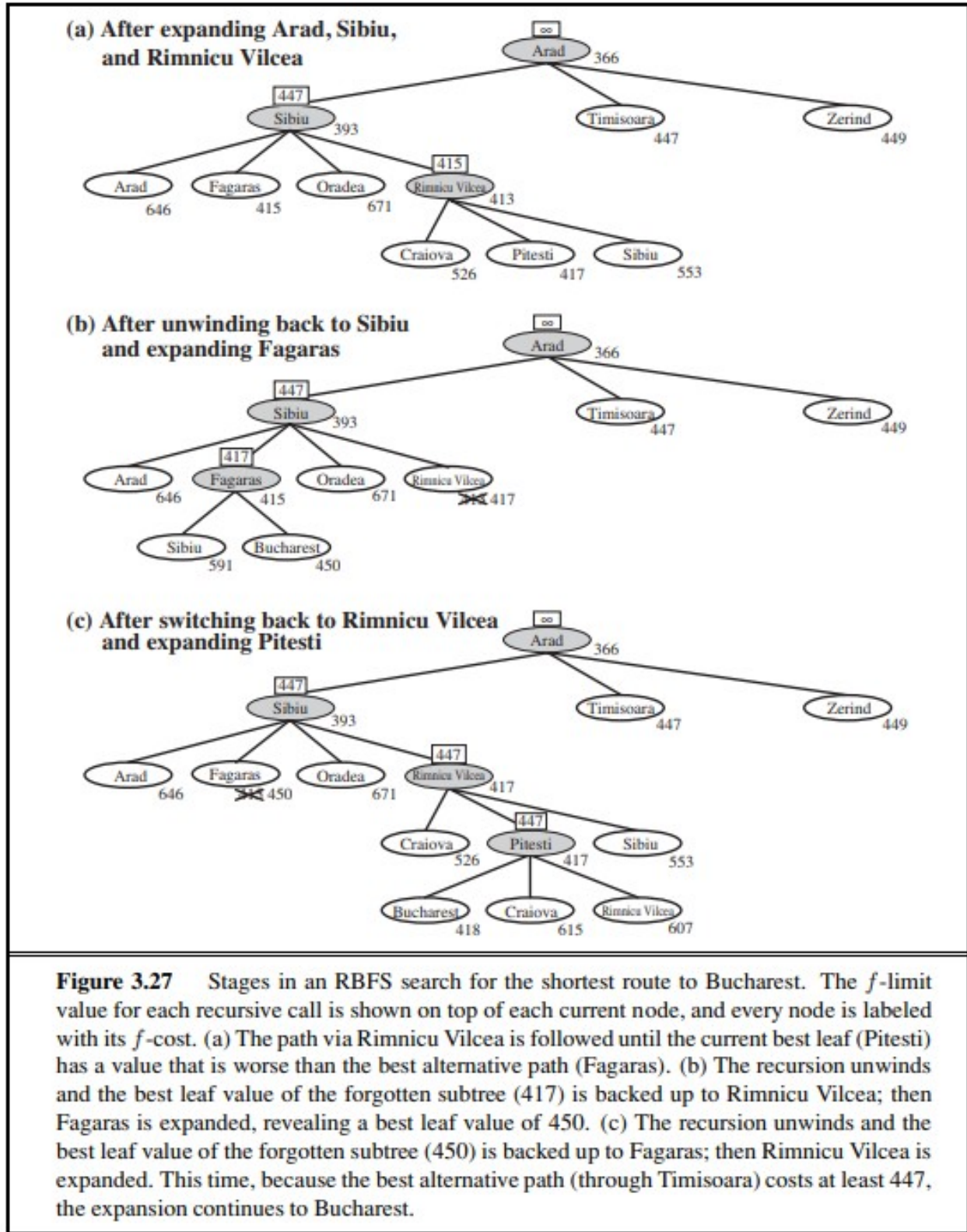
3.5.3 Bellek Sınırlı Sezgisel Arama

IDA*

Iterative-Deepening A* (IDA*) klasik iterative deepening fikrini heuristic aramaya taşır. Derinlik limiti yerine $f=g+h$ limiti kullanılır. Her turda, önceki turda limiti aşan en küçük f değeri yeni cutoff olur. Çok daha az bellek kullanır; ancak düğümler yeniden üretilebilir.

Recursive Best-First Search (RBFS)

RBFS standart best-first davranışını doğrusal bellekle taklit etmeye çalışır. Rekürsif DFS'ye benzer; fakat mevcut yoldan sonsuza dek ilerlemek yerine ataların en iyi alternatif yolunun f değerini f -limit içinde tutar. Mevcut en iyi yaprak bu limiti aşarsa recursion geri sarar ve alternatif yola geçer. Unutulan alt ağacın en iyi yaprak f değeri parent'a backed-up value olarak aktarılır.



Şekil 3.27 - Bükreş'e en kısa rota için RBFS aşamaları. Her aktif düğümün üzerinde recursive f -limit, düğümün yanında f değeri gösterilir.

Örnekte Rimnicu Vilcea 413 ile Fagaras 415'ten iyi görünür ve önce seçilir. Pitesti'ye gelindiğinde en iyi yaprak 417'ye yükselir; bu değer alternatif Fagaras 415'ten kötüdür. RBFS geri döner ve Rimnicu alt ağacını 417 değerleriyle hatırlar. Fagaras genişletildiğinde en iyi yaprak 450 olur; bu kez 417'lik Rimnicu yolu yeniden daha iyi hale gelir. Algoritma Rimnicu'ya döner, Pitesti üzerinden devam eder ve Bucharest 418'e ulaşır.

RBFS admissible heuristic ile A* Tree Search gibi optimaldir ve belleği çözüm derinliğiyle doğrusal büyür. Dezavantajı, unuttuğu düğümleri tekrar tekrar oluşturmasıdır.

SMA*

SMA* (Simplified Memory-Bounded A*) mevcut belleğin tamamını kullanır. Bellek dolana kadar A* gibi ilerler; yeni düğüm eklemek için yer kalmadığında en yüksek f değeri, yani en kötü yaprak silinir ve değeri parent'a yedeklenir. Daha sonra diğer yollar kötüleşirse silinen alt ağaç yeniden üretilebilir. En sık çözüm bellek kapasitesinin içinde kalıyorsa complete'tir; optimal çözüm erişilebiliyorsa optimaldir. Bellek çok kısıtlıysa sürekli alt ağaç silme-yeniden üretme thrashing'e yol açabilir.

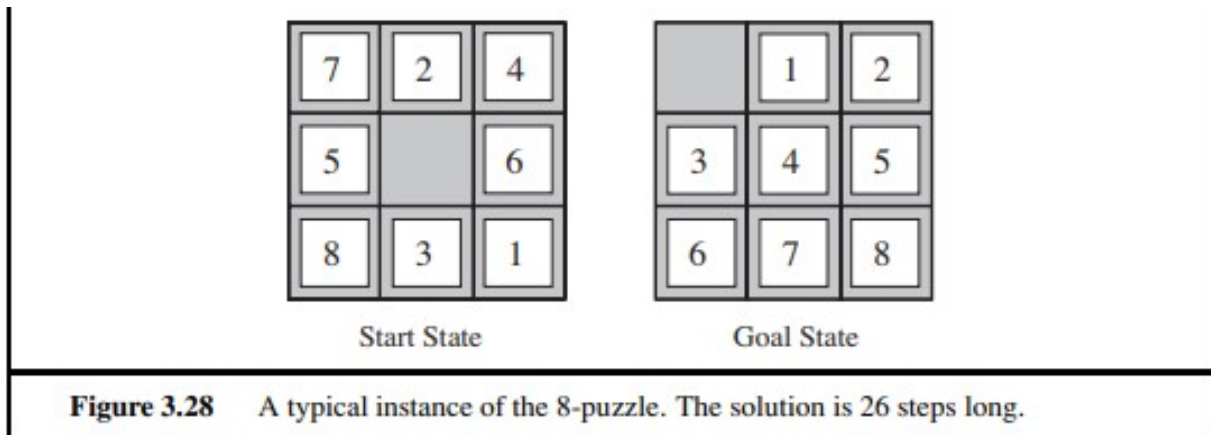
3.5.4 Daha İyi Aramayı Öğrenmek

Bir ajan yalnızca önceden tasarlanmış BFS, Greedy veya A* stratejilerini kullanmak zorunda değildir; arama sürecinden öğrenebilir. Object-level state space asıl problemin durum uzayıdır (örneğin Romanya şehirleri). Metalevel state space ise arama programının kendi iç hesaplama durumlarını temsil eder; örneğin A*'ın o anda oluşturmuş olduğu search tree bir metalevel state olabilir.

Bir yaprağın genişletilmesi, metalevel uzayda bir hesaplama eylemidir. Ajan geçmiş aramalarda işe yaramayan genişletmelerden öğrenerek gelecekte umut vermeyen alt ağaçlardan kaçınabilir. Metalevel learning'in amacı, hesaplama maliyeti ile bulunan yolun maliyetini dengeleyerek toplam problem çözme maliyetini azaltmaktır.

3.6 SEZGİSEL FONKSİYONLAR

Bu bölümde heuristic fonksiyonların doğasını açıklamak için yeniden 8-Puzzle kullanılır. Rastgele bir 8-Puzzle probleminin ortalama çözüm maliyeti yaklaşık 22 adımdır; branching factor yaklaşık 3'tür. Kör tree search $3^{22} \approx 3,1 \times 10^{10}$ durumu inceleyebilir. Graph Search tekrarları engelleyerek 181.440 erişilebilir duruma düşürse de 15-Puzzle gibi daha büyük problemlerde iyi bir heuristic zorunludur.



Şekil 3.28 - Tipik bir 8-Puzzle örneği. Bu belirli problemin optimal çözümü 26 adımdır.

h1: Yanlış konumdaki taş sayısı

h1, hedef konumunda olmayan taşların sayısıdır. Şekil 3.28'de sekiz taşın tamamı yanlış yerde olduğundan h1=8'dir. Her yanlış taşın en az bir kez hareket etmesi gerektiği için h1 gerçek çözüm maliyetini aşamaz ve admissible'dir.

h2: Manhattan Distance

h2, her taşın mevcut konumundan hedef konumuna yatay ve dikey kare farklarının toplamıdır. Çapraz hareket olmadığı için Manhattan (city-block) distance kullanılır:

$$\text{Manhattan}(\text{tile}) = |\Delta \text{row}| + |\Delta \text{column}|$$

Şekil 3.28'de taşların katkıları $3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18$ 'dir. Gerçek optimal maliyet 26 olduğundan h1=8 ve h2=18 değerlerinin ikisi de gerçek maliyeti aşmaz. h2, 26'ya daha yakın olduğu için daha bilgilendiricidir.

3.6.1 Heuristic Doğruluğunun Performansa Etkisi

Heuristic kalitesini ölçmenin bir yolu effective branching factor b^* değeridir. A^* belirli bir problemde N düğüm üretip çözümü d derinliğinde bulmuşsa, b^* şu eşitliği sağlayan dallanma faktörüdür:

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

Örneğin A^* $d=5$ çözümünü $N=52$ düğümle buluyorsa b^* yaklaşık 1,92'dir. İyi heuristic için b^* değerinin 1'e yakın olması istenir; bu durumda arama neredeyse doğrudan doğruya çözüm yoluna odaklanır.

d	Search Cost (nodes generated)			Effective Branching Factor		
	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	—	539	113	—	1.44	1.23
16	—	1301	211	—	1.45	1.25
18	—	3056	363	—	1.46	1.26
20	—	7276	676	—	1.47	1.27
22	—	18094	1219	—	1.48	1.28
24	—	39135	1641	—	1.48	1.26

Figure 3.29 Comparison of the search costs and effective branching factors for the ITERATIVE-DEEPENING-SEARCH and A^* algorithms with h_1 , h_2 . Data are averaged over 100 instances of the 8-puzzle for each of various solution lengths d .

Şekil 3.29 - IDS, $A^*(h_1)$ ve $A^*(h_2)$ için arama maliyetleri ve effective branching factor değerleri. Her d değeri için 100 rastgele 8-Puzzle örneğinin ortalaması alınmıştır.

Tablo, h_2 'nin h_1 'den ve bilgisiz IDS'den belirgin biçimde daha iyi olduğunu gösterir. $d=12$ 'de IDS ortalama 3.644.035 düğüm, $A^*(h_1)$ 227 düğüm, $A^*(h_2)$ yalnızca 73 düğüm üretmiştir. Aynı satırda effective branching factor yaklaşık 2,78; 1,42; 1,24'tür.

Dominance (Baskınlık)

Her n için $h_2(n) \geq h_1(n)$ ve iki heuristic de admissible ise h_2 , h_1 'i dominate eder. $A^*(h_2)$, $f=C^*$ üzerindeki eşitlikler dışında $A^*(h_1)$ 'den daha fazla düğüm genişletmez. Bu yüzden admissibility/consistency korunuyor ve heuristic hesaplama maliyeti aşırı değilse daha yüksek, gerçeğe yakın alt sınır genellikle daha iyidir.

3.6.2 Gevşetilmiş Problemlerden Admissible Heuristic Üretmek

Bir problemin bazı eylem kısıtları kaldırılarak daha kolay bir relaxed problem oluşturulabilir. Relaxed problem orijinalin state-space grafiğine ek kenarlar eklediği için relaxed optimum, orijinal optimumdan daha pahalı olamaz. Bu nedenle relaxed problemin optimal çözüm maliyeti, orijinal problem için admissible heuristic olur; ayrıca tam relaxed maliyet üçgen eşitsizliğini sağladığından consistent olur.

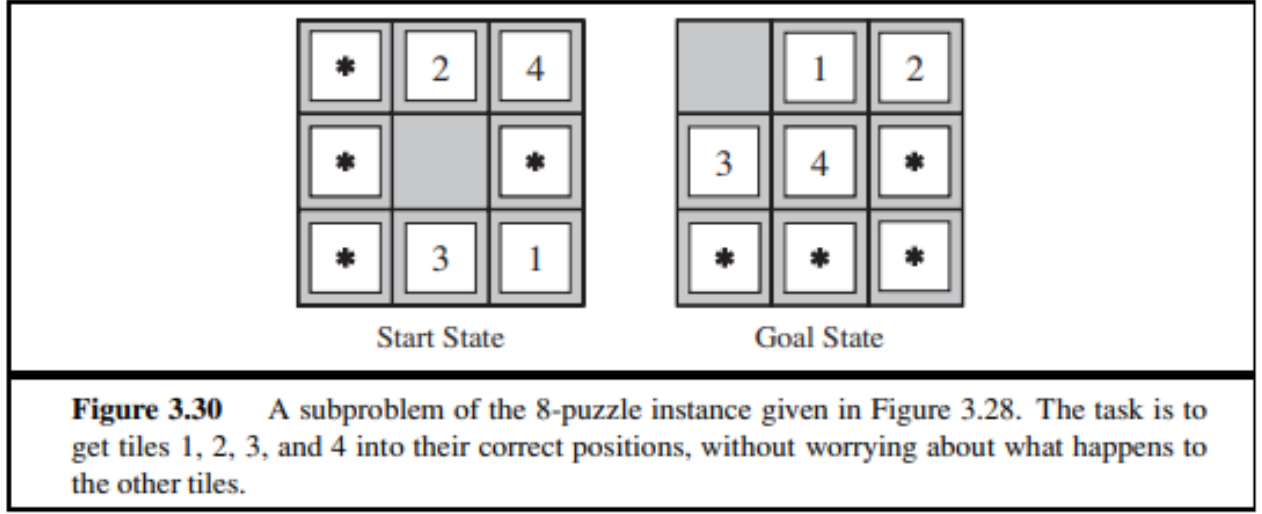
8-Puzzle'nın normal kuralı: "Bir taş A karesinden B'ye, A ve B yatay/dikey komşuysa ve B boşsa hareket edebilir." Bir koşul kaldırılırsa "komşuysa hareket et" kuralı Manhattan distance h_2 'yi doğurur. Her iki koşul kaldırılıp bir taşın herhangi bir kareden herhangi bir kareye gitmesine izin verilirse misplaced tiles h_1 elde edilir.

Birden fazla admissible heuristic varsa ve hiçbirini her yerde dominate etmiyorsa güvenli bir bileşik heuristic kullanılabilir:

$$h(n) = \max\{h_1(n), h_2(n), \dots, h_m(n)\}$$

Bileşenlerin her biri admissible olduğundan maksimumları da gerçek maliyeti aşmaz; ayrıca bu birleşik heuristic tüm bileşenlerini dominate eder. Ancak heuristic'in kendisini hesaplamak çok pahalıysa daha doğru tahmin her zaman daha hızlı toplam çözüm anlamına gelmeyebilir.

3.6.3 Alt Problemlerden Heuristic Üretmek: Pattern Databases



Şekil 3.30 - Şekil 3.28'deki 8-Puzzle'in alt problemi. Amaç yalnızca 1, 2, 3 ve 4 taşlarını doğru konuma getirmektir; diğer taşların konumu önemsenmez.

Bir alt problemin optimal maliyeti, tam problemin optimal maliyeti için alt sınırdır. Örneğin yalnızca 1,2,3,4 taşlarını yerine getirmek en az 10 hareket gerektiriyorsa bütün puzzle 10 hareketten daha azıyla çözülemez. Bu nedenle alt problem maliyeti admissible heuristic olabilir.

Pattern database, seçilmiş taş grubunun bütün olası konfigürasyonları için gerçek optimal çözüm maliyetlerini önceden hesaplayıp saklar. Arama sırasında tam state geldiğinde ilgili pattern veritabanından doğrudan lookup yapılarak h_DB elde edilir. Veritabanı genellikle hedeften geriye doğru arama yapılarak oluşturulur; pahalı ön-hesaplama daha sonra çok sayıda puzzle örneğine yayılır.

1-2-3-4, 5-6-7-8 veya 2-4-6-8 gibi farklı pattern'ler ayrı admissible heuristic'ler üretir ve maksimum alınarak birleştirilebilir. Kaynakta birleşik pattern database heuristic'lerinin 15-Puzzle'da Manhattan distance'a kıyasla üretilen düğüm sayısını yaklaşık 1000 kat azaltabildiği belirtilir.

Disjoint Pattern Databases

İki pattern maliyetini doğrudan toplamak, aynı fiziksel hamlenin her iki alt problemde de sayılması nedeniyle overestimate oluşturabilir. Disjoint pattern database yaklaşımında her alt probleme yalnızca kendi taş grubunu hareket ettiren hamlelerin maliyeti yazılır; böylece alt maliyetler güvenle toplanabilir. Sliding-tile puzzle'larda bu, 15-Puzzle için yaklaşık 10.000 kat, 24-Puzzle için yaklaşık bir milyon kat hızlanma sağlayabilen çok güçlü heuristic'ler üretebilir.

3.6.4 Deneyimden Heuristic Öğrenme

Heuristic'ler yalnızca elle tasarlanmak zorunda değildir. Bir ajan çok sayıda problem çözerek deneyimden $h(n)$ öğrenebilir. Her optimal çözüm yolu üzerindeki state, o state'ten hedefe gerçek kalan maliyetle birlikte bir eğitim örneği verir. Neural network, decision tree ve reinforcement learning gibi yöntemlerle yeni durumların çözüm maliyeti tahmin edilebilir.

Ham state yerine anlamlı özellikler kullanmak faydalı olabilir. Örneğin $x1(n)$ yanlış yerdeki taş sayısı, $x2(n)$ ise hedefte komşu olması gerekirken mevcut durumda komşu olmayan taş çiftlerinin sayısı olsun. Basit doğrusal model:

$$h(n) = c1 \cdot x1(n) + c2 \cdot x2(n)$$

c1 ve c2 gerçek çözüm maliyeti verilerine en iyi uyacak biçimde öğrenilir. Böyle bir model hedefte $h=0$ koşulunu sağlayabilir; ancak öğrenilmiş heuristic'in admissible veya consistent olması otomatik olarak garanti edilmez. Daha iyi tahmin doğruluğu ile A*'ın teorik optimality garantisi farklı hedeflerdir.

ÜNİTE 3 - HIZLI TEKRAR

Kavram	Kısa tanım
State	Dünyanın bir konfigürasyonu
Node	Search tree içinde state + parent + action + path-cost taşıyan veri yapısı
Frontier	Üretilmiş ama henüz genişletilmemiş düğümler
Explored Set	Daha önce genişletilmiş durumlar
$g(n)$	Başlangıçtan n'e gerçek maliyet
$h(n)$	n'den hedefe tahmini maliyet
$f(n)$	Düğüm değerlendirme değeri; A* için $g+h$
Admissible	Heuristic gerçek kalan maliyeti asla aşmaz
Consistent	$h(n) \leq c(n,a,n') + h(n')$
Dominance	İki admissible heuristic'ten biri her state'te daha yüksek/eşitse daha bilgilendiricidir

Algoritma	Seçim kuralı	Veri yapısı	Öne çıkan özellik
BFS	En sığ	FIFO	Complete; eşit maliyette optimal; bellek pahalı
UCS	En düşük g	Priority Queue	Genel maliyetlerde optimal
DFS	En derin	LIFO/Stack	Çok az bellek; optimal değil
DLS	En derin, limitli	Stack/Recursion	Sonsuz derinliği sınırlar
IDS	Artan depth limit	Tekrarlı DFS	BFS tamlığı + DFS belleği
Greedy	En düşük h	Priority Queue	Hızlı olabilir; optimal değil
A*	En düşük $g+h$	Priority Queue	Uygun heuristic ile complete/optimal
IDA*	Artan f limiti	DFS benzeri	A* mantığı, düşük bellek
RBFS	En iyi f + alternatif limit	Recursion	Doğrusal bellek
SMA*	A* + bellek sınırı	Memory-bounded frontier	Bellek dolunca en kötü yaprağı siler

En önemli üç formül

Uniform-Cost: $f(n)=g(n)$ | Greedy: $f(n)=h(n)$ | A*: $f(n)=g(n)+h(n)$

Bu üniteyi çalışırken kavramları yalnızca ezberlemek yerine her algoritmada aynı dört soruyu sorun: (1) Frontier'dan hangi düğümü seçiyor? (2) Complete mi? (3) Optimal mi? (4) Zaman ve bellek maliyeti nedir? Bu çerçeveye, bütün arama yöntemlerini birbirine bağlar.

--- Ünite 3 sonu ---